Year: 2026-2027 **Sem V-NEP**

Course: Agile Software Development & DevOps Lab

Class: T1 **Date of Submission: 11/10/2026**

MINI PROJECT REPORT ON

"AI Resume Analyzer: An ATS-Style Resume Scoring Application Delivered Through an Agile and DevOps Pipeline"

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS OF DEGREE OF

BACHELOR OF ENGINEERING BY

Kunal Dodechani (T1,2401025)

Mohit Gabra (T1,2401027)

Deepa Garg (T1,2401029)

SUBJECT

Agile Software Development and DevOps Lab

SUPERVISOR

Prof. Sanober Shaikh



CERTIFICATE

This is to certify that the mini project entitled **"AI Resume Analyzer: An ATS-Style Resume Scoring Application Delivered Through an Agile and DevOps Pipeline"** is a bona fide work of **Kunal Dodechani (2401025)**, **Mohit Gabra (2401027)**, **Deepa Garg (2401029)**,submitted in partial fulfilment of the requirements of the course Agile Software Development & DevOps Lab, Semester V, during the academic year 2026–2027.

_____ _____

Project Guide **Head of Department**

Date: _____ Place: _____

ABSTRACT

Recruiters increasingly use Applicant Tracking Systems (ATS) that filter resumes by keyword before any person reads them, and candidates usually have no way of knowing how well a resume matches a given job description. The AI Resume Analyzer addresses this with a web application in which a candidate pastes or uploads a resume (PDF, DOCX or text), supplies the target job description, and receives an instant analysis: an overall match score, the technical skills that are matched, the skills that are missing, a skill matrix grouped by technology domain, and actionable recommendations for improving the resume.

The application served as the vehicle for applying all seven experiments of the Agile Software Development & DevOps Lab to one product. The work was planned in Jira as a Scrum backlog and sprint and linked to GitHub, so that issue keys appear in branches, commits and pull requests. Every change was versioned with Git and merged through GitHub. The Flask application was packaged as a Docker image and, together with nginx, MySQL, Redis and phpMyAdmin, defined in a single Docker Compose file. Jenkins was installed to run Declarative and Scripted pipelines with Build, Test and Deploy stages, and a Jenkinsfile stored in GitHub allowed Jenkins to fetch and run the pipeline from source control. GitHub Actions runs the pytest tests on every push and pull request. Finally, Nagios Core running in a Docker container monitors a remote Windows host through the NSClient++ agent.

The work was divided among three members, each of whom performed part of every experiment on their own machine and GitHub account, and evidence in the form of command output and screenshots is included for every experiment. The project shows that Agile planning, version control, containers, continuous integration, automated deployment and monitoring are most useful when they operate together as one delivery flow: Jira → Git and GitHub → Docker and Docker Compose → Jenkins → Nagios.

Keywords: Agile, Scrum, Jira, Git, GitHub, Docker, Docker Compose, Jenkins, CI/CD, GitHub Actions, Nagios, NSClient++, Flask, Python

TABLE OF CONTENTS

Contents		Page Numbers
1.	Introduction	5
2.	Requirements Analysis	7
3.	System Design	8
4.	Application Implementation	10
5.	DevOps Implementation (Experiments 1–7)	13
6.	Testing	59
7.	Results and Discussion	60
8.	Conclusion and Future Scope	61
9.	References	62
10.	Appendix: Running the Project	62
11.	Lab Outcomes	63

1. Introduction

1.1 Background

A job opening in a technical field routinely attracts hundreds of resumes. To cope with the volume, recruiters use Applicant Tracking Systems that score each resume against the job description, mostly by looking for the technical keywords the employer asked for. A well-qualified candidate can therefore be rejected simply because the resume does not use the vocabulary of the posting, and the candidate rarely learns why.

The Agile Software Development & DevOps Lab introduced, through seven separate experiments, the practices used to deliver software of this kind reliably: version control, containers, multi-container orchestration, continuous integration, automated deployment, Agile project tracking and monitoring. Each experiment was first performed in isolation on a small workload. This mini project applies all seven to one realistic application, the AI Resume Analyzer, so that the way the tools fit together becomes visible, rather than each tool being seen on its own.

1.2 Problem Statement

To design, build and operate a web application that analyses a resume against a target job description and reports a match score, matched and missing skills and recommendations, and to deliver it through an Agile, automated pipeline in which work is planned in Jira, versioned in Git and GitHub, packaged with Docker and Docker Compose, built and deployed by Jenkins, tested by GitHub Actions and observed by Nagios.

1.3 Objectives

1. Allow a candidate to paste or upload a resume and a job description and receive an instant match analysis.
2. Report the match score, matched keywords, missing key technical skills, a skill matrix by technology domain and recommendations.
3. Plan the work as a Jira Scrum backlog and sprint, with issue keys linking commits and pull requests to tickets.
4. Place the project under Git, publish it to GitHub, and practise branching, merging, conflict resolution, reset, revert and restore.
5. Package the application as a Docker image and run it as a container.
6. Run the application together with nginx, MySQL, Redis and phpMyAdmin from one Docker Compose file.
7. Install Jenkins, build Declarative and Scripted pipelines, and run a pipeline whose Jenkinsfile is fetched from GitHub.
8. Monitor a remote Windows host with Nagios and the NSClient++ agent.

1.4 Scope

The system analyses one resume against one job description at a time. Analysis is rule-based: a technical taxonomy of more than sixty skills across five domains is matched with regular

expressions and scored with weights, which keeps results fast and repeatable and needs no external service. Results are cached in Redis with an in-memory fallback when Redis is not available. The deployment target is a developer machine running Docker; the same Dockerfile, Compose file and Jenkinsfile would apply to a college server. The Deploy stage of the Jenkins pipeline is demonstrated with echo steps, as in the lab experiments.

1.5 Mapping of Lab Experiments to the Project

Exp	Lab topic	Application in the AI Resume Analyzer	Section
1	Version control with Git	Repository ai-resume-analyzer, feature branches, merge, merge conflict, reset, revert and restore	5.1
2	Containerisation with Docker	Dockerfile and image resume-analyzer:v1; 25 Docker commands	5.2
3	Docker Compose	compose-demo stack: web (nginx), app, database, redis, phpmyadmin	5.3
4	Jenkins CI pipeline	Declarative and Scripted pipelines with Build, Test and Deploy stages	5.4
5	Automated deployment	Jenkinsfile in GitHub, Pipeline script from SCM, GitHub hook trigger	5.5
6	Agile project lifecycle	Jira Scrum project ARA, sprint, GitHub for Jira, automation rules, GitHub Actions	5.6
7	Monitoring	Nagios Core in Docker monitoring a Windows host through NSClient++	5.7

Table 1.1: Lab experiments and where each is applied

1.6 Team Roles and Work Division

The group has three members. Every member performed part of every experiment on their own PC, with their own GitHub account, and took screenshots of their own commands. Deepa Garg is the Foundation / Owner, who creates the project, the GitHub repository, the Jira space and the Jenkins and Nagios servers. Kunal Dodechani is the Builder, who develops the first feature branch, runs Docker commands 10 to 17, adds the database services to Compose, builds the Declarative pipeline, handles the Jira smart commit and pull request, and sets up the Nagios remote agent. Mohit Gabra is the Integrator, who develops the second feature branch and the merge conflict, runs Docker commands 18 to 25, adds the redis service, builds the Scripted pipeline, connects Jenkins to GitHub, configures the Jira merge rule and conflict, and adds the host in Nagios.

Experiment	Deepa Garg	Kunal Dodechani	Mohit Gabra
EXP 01 Git	Steps 1–13 (setup, first push); 31, 32 (history, branches); 35 (restore)	Steps 14–19 (resume upload branch); 26 (merge); 33 (soft reset)	Steps 20–25 (dashboard branch); 27–30 (merge conflict); 34 (revert)
EXP 02 Docker	Installation 1–6; commands 1–9; Dockerfile and image build	Installation 1–6; commands 10–17 and Linux commands	Installation 1–6; commands 18–25
EXP 03 Compose	web and app services; down, up -d, config	database and phpmyadmin services; up -d, ps, logs, images	redis service; stop, start, restart, down, down -v, config
EXP 04 Jenkins CI	Procedure 1–3 (JDK, jenkins.war, unlock, plugins, admin); create users	Procedure 4 and 5: Declarative pipeline	Procedure 4 and 5: Scripted pipeline
EXP 05 Jenkins + GitHub	Part 1 (verify Jenkins); Part 4 (Jenkinsfile in GitHub)	Part 2 (first job); Part 3 (run locally); webhook	Part 5 (connect Jenkins to GitHub); Part 6 (verify build)
EXP 06 Jira	Steps 1–5; 9, 10 (GitHub Actions); 12 (sprint completion)	Step 6 (smart commit); 7 (rule: In Review); 8 (pull request); 11 (review and merge)	Step 7 (rule: Done); 8 (review and merge); 11 (second branch, conflict)
EXP 07 Nagios	Steps 1–4 (Nagios server); 7 (verify)	Steps 5–6 (NSClient++ on the remote host)	Steps 8–9 (add host, check in web interface)

Table 1.2: Master distribution of experiment steps among the members

2. Requirements Analysis

This chapter records the requirements of the AI Resume Analyzer: who uses it, what it must do and the qualities it must have. The work itself was planned and tracked as a Scrum project in Jira, which is documented together with the other experiments in Chapter 5 (Section 5.6).

2.1 User Roles

Role	Goal	Main screens
Candidate	Find out how well a resume matches a job description and how to improve it	Input form (paste or upload), results dashboard
Recruiter / reviewer	Compare resume content with the technical requirements of a posting	Results dashboard, exported JSON report
Operator	Keep the service available and observe its health	/health endpoint, Nagios web interface

Table 2.1: User roles

2.2 Functional Requirements

- The user can paste resume text or upload a resume file in PDF, DOCX, DOC, TXT or Markdown format.
- The user can paste a target job description, or leave it empty to benchmark against a built-in profile; sample profiles (for example DevOps / SRE, Full-Stack Engineer, Data / ML Engineer) can be loaded with one click.
- The application returns a candidate match score as a percentage, together with the matched skills and the missing key skills.
- A skill matrix groups the detected skills by technology domain such as languages and AI & data science.
- Actionable recommendations are generated from the skill gaps.
- The results can be exported as JSON, and the form can be reset for a new analysis.
- GET /health returns HTTP 200 with the status of the service and its cache.

2.3 Non-Functional Requirements

- **Performance:** repeated analyses of the same resume and job description are served from a cache keyed by a SHA-256 hash of the input.
- **Resilience:** if Redis is unavailable, a thread-safe in-memory dictionary is used, so the application keeps working (the health response reports the mode).
- **Portability:** the application starts with python app/app.py on port 5000, in a Docker container, or with docker compose up -d behind nginx.
- **Maintainability:** automated pytest tests run on every push and pull request through GitHub Actions.
- **Observability:** the /health endpoint can be checked by Nagios and by Docker, and the host is monitored through NSClient++.

3. System Design

3.1 Architecture

The AI Resume Analyzer follows a layered architecture. The presentation layer is a single-page application served from templates/index.html. Requests reach nginx on port 8081, which acts as a reverse proxy and passes them to the Flask backend on port 5000. In the backend, an HTTP router exposes GET /, GET /health and POST /analyze. An uploaded document goes through a document ingestion stage, whose parsers depend on the file type, and the normalised text is passed to the core analysis engine. The result is stored by a cache manager that uses Redis as the primary cache and a thread-safe in-memory dictionary as fallback. Jenkins and Docker form the containerisation and CI/CD layer that builds, tests and deploys the backend.

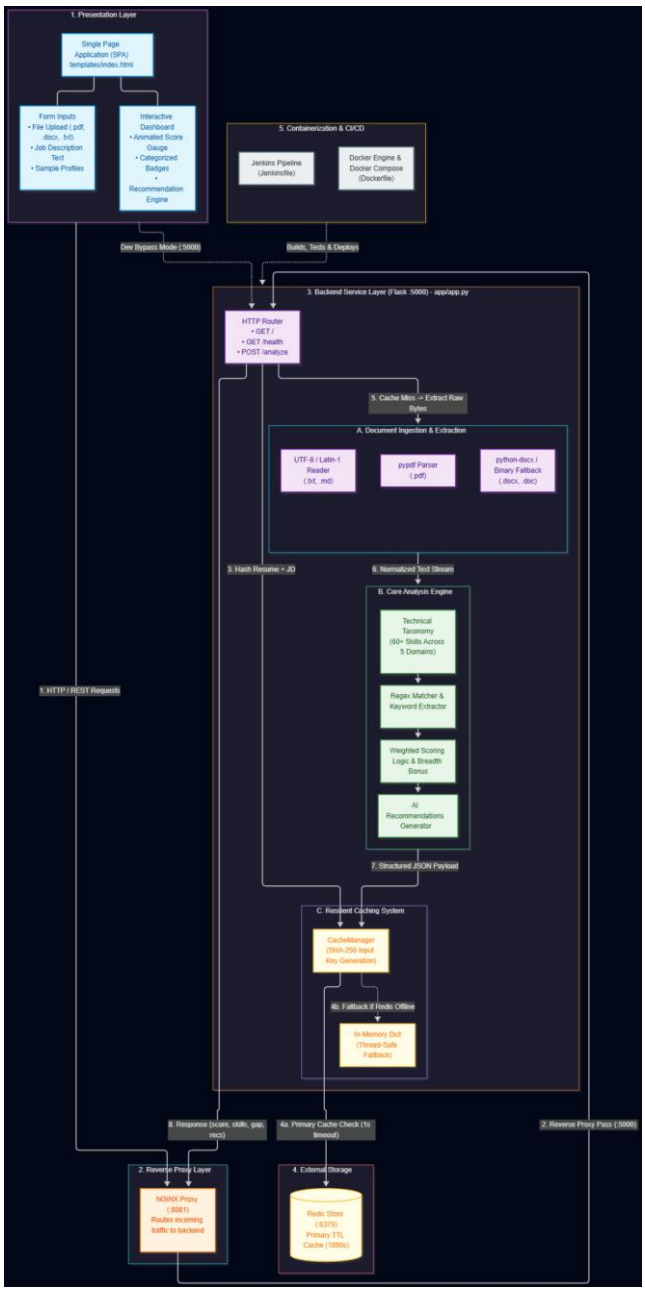


Figure 3.1: System architecture of the AI Resume Analyzer

3.2 Components

Layer	Component	Responsibility
Presentation	templates/index.html (SPA)	Input form (file upload, job description, sample profiles) and interactive dashboard with animated score gauge, categorised badges and recommendations
Reverse proxy	nginx (port 8081)	Routes incoming traffic to the backend
Backend	Flask app/app.py (port 5000)	HTTP router with GET /, GET /health and POST /analyze

Layer	Component	Responsibility
Ingestion	Text reader, pypdf, python-docx	Extracts text from .txt/.md, .pdf and .docx/.doc files
Analysis	Technical taxonomy, regex matcher, scoring, recommendations	Detects 60+ skills across 5 domains, computes a weighted score with a breadth bonus and generates recommendations
Caching	CacheManager, Redis (port 6379), in-memory dictionary	SHA-256 key of resume and job description; Redis TTL of 1800 s; in-memory fallback if Redis is offline
DevOps	Docker, Docker Compose, Jenkins, GitHub Actions, Nagios	Packaging, orchestration, build, test, deployment and monitoring

Table 3.1: Components of the system

3.3 Ports and Machines

Service	Port	Where it runs
Application (Flask)	5000	Each member's own PC
Jenkins	8080	Deepa Garg's PC (Kunal Dodechani and Mohit Gabra open it over the network)
Compose nginx (web)	8081	Each member's own PC
Compose phpMyAdmin	8082	Each member's own PC
Nagios web interface	8090	Deepa Garg's PC (8090 is used because Jenkins already uses 8080)
NSClient++ (NRPE)	5666	Kunal Dodechani's PC, the remote Windows host that is monitored

Table 3.2: Ports and machines used in the project

4. Application Implementation

4.1 Resume and Job Description Input

The first screen has two panels. In the Candidate Resume panel the user pastes resume content or switches to Upload File and drops a PDF, DOCX, DOC, TXT or Markdown file. In the Target Job Description panel the user pastes the job posting, or leaves it empty to benchmark against a built-in profile. Quick-load samples (DevOps / SRE, Full-Stack Engineer, Data / ML Engineer) fill the form with one click. The system status indicator in the header shows System: UP and the cache mode.

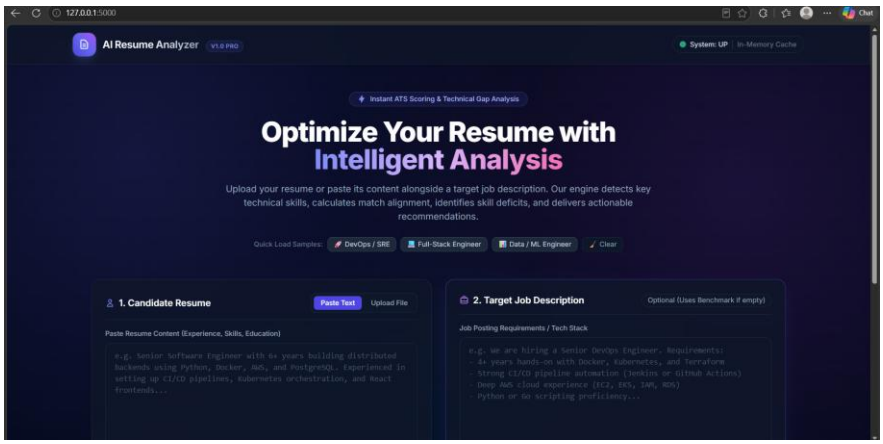


Figure 4.1: Landing page of the AI Resume Analyzer with the paste-text form

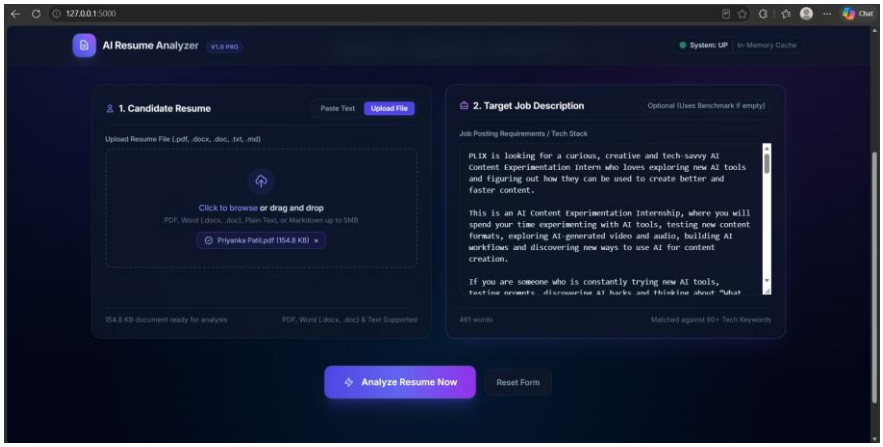


Figure 4.2: Uploading a resume file and entering the target job description

4.2 Analysis Results Dashboard

After Analyze Resume Now is pressed, the results dashboard shows an animated gauge with the candidate match score, the number of matched skills and the number of missing skills, the matched keywords and the missing key technical skills, a skill matrix by technology domain, and actionable recommendations. The results can be exported as JSON.

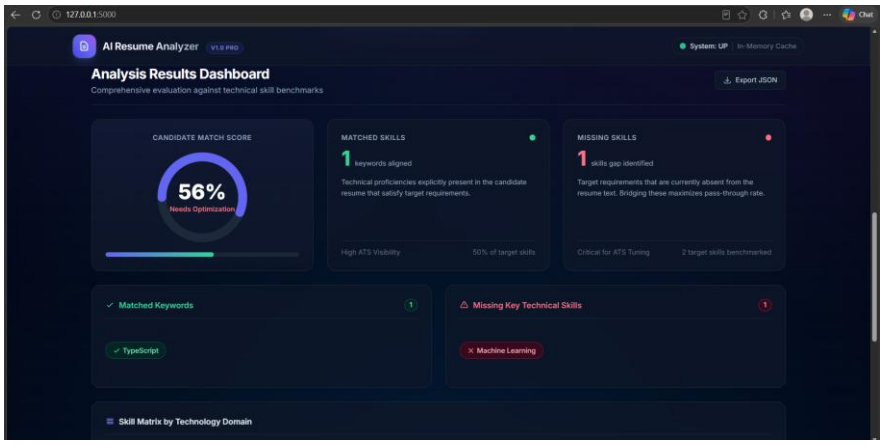


Figure 4.3: Results dashboard with the candidate match score, matched skills and missing skills

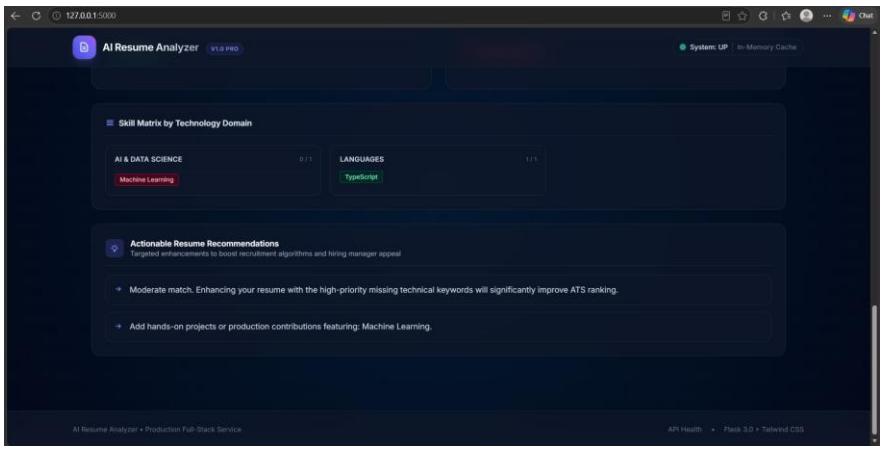


Figure 4.4: Skill matrix by technology domain and actionable resume recommendations

4.3 Analysis Pipeline

1. The resume and job description are hashed with SHA-256, and the cache is checked first (Redis, then the in-memory fallback).
2. On a cache miss, the raw bytes of an uploaded file are passed to the ingestion stage: a UTF-8 / Latin-1 reader for text files, pypdf for PDF files and python-docx (with a binary fallback) for Word files.
3. The normalised text is matched against the technical taxonomy by a regular-expression matcher and keyword extractor.
4. A weighted scoring function with a breadth bonus computes the match score, and the recommendations generator turns the missing skills into suggestions.
5. The structured JSON payload (score, skills, gaps and recommendations) is stored in the cache and returned to the browser.

4.4 Health Endpoint

GET /health returns a small JSON document with the status of the service, the cache mode, and whether Redis is connected. It is used to confirm that the application runs locally, in a container and behind nginx, and it is the target of the automated tests and the monitoring checks described in later chapters.

5. DevOps Implementation

This chapter documents Experiments 1 to 7 as applied to the AI Resume Analyzer. For each experiment it gives the aim, the theory, the procedure with the commands, the output captured by each member, and a conclusion.

5.1 Version Control with Git and GitHub (Experiment 1)

5.1.1 Aim

To perform version control operations using Git (LO2: perform version control operations using Git and apply collaboration workflows). On the AI Resume Analyzer the project is placed under Git and pushed to GitHub, two feature branches are developed by two members and merged, a merge conflict is created and resolved, and rollback, revert and restore are practised.

5.1.2 Theory

Version control records changes to files so that earlier versions can be recalled and several people can work on one project. Git is distributed: every developer holds the full repository and its history, and shares commits through a remote such as GitHub. Changes move from the working directory to the staging area (git add) and into the repository as a commit; a branch isolates a feature, and merging joins branches with a fast-forward or a merge commit. When two branches change the same lines, Git reports a merge conflict and marks the file with <<<<<<<, ======= and >>>>>>>, which the developer removes after choosing the final content.

Work is shared with a remote (origin) using git clone, push, fetch and pull. Mistakes are undone with git reset --soft HEAD~1 (drop the last commit but keep the changes staged), git revert (a new commit that reverses an earlier one, safe for shared branches) and git restore (discard uncommitted changes).

Command	Purpose
git init / git clone	Create a new repository / copy an existing one
git config	Set user name, email and other settings
git status / git add / git commit	Show changes / stage changes / record a snapshot
git branch / git checkout -b	List branches / create and switch to a branch
git merge	Join another branch into the current one
git push / git pull / git fetch	Upload / download and merge / download commits
git log --oneline --graph --all	Show the commit history with branch visualisation
git reset --soft / git revert / git restore	Roll back a commit / reverse a commit by a new one / discard uncommitted changes

Table 5.1: Git commands used in Experiment 1

5.1.3 Implementation

Steps 1 to 13

```
mkdir ai-resume-analyzer
cd ai-resume-analyzer
git init
git config --global user.name "Your Name"
git config --global user.email "your-email@example.com"
git config --list
echo "AI Resume Analyzer Collaboration Project" > project.txt
git status
git add .
git commit -m "Initial commit"
git branch -M main
git remote add origin https://github.com/<gh-user>/ai-resume-analyzer.git
git remote -v
git push -u origin main
```

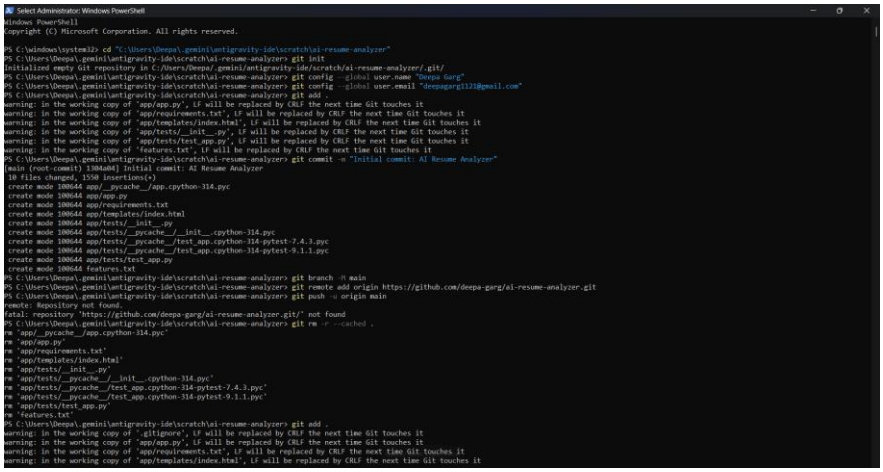


Figure 5.1: Git initialisation, configuration, first commit and remote setup (Steps 2–12)

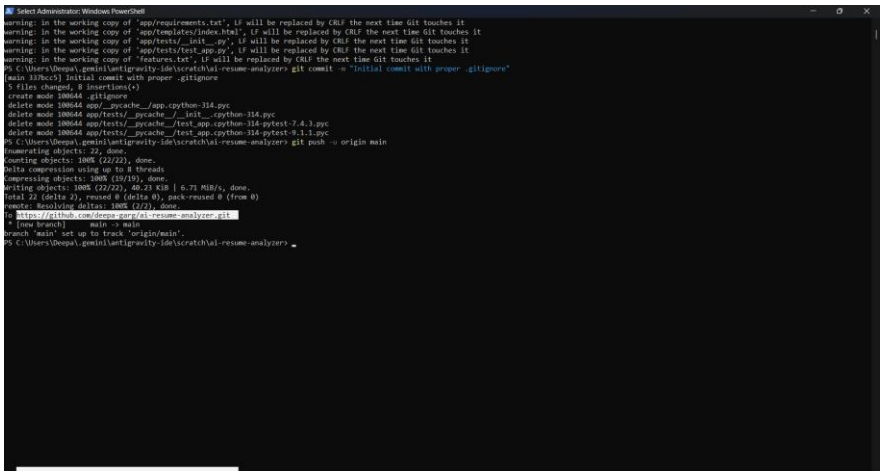


Figure 5.2: Commit, branch rename and push of main to GitHub (Steps 9–13)

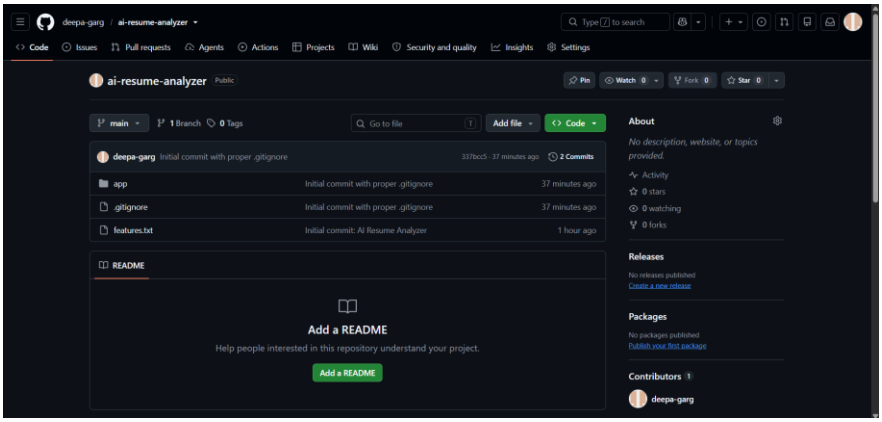


Figure 5.3: Repository ai-resume-analyzer on GitHub after the first push

Steps 14 to 19 and 26

```
git clone https://github.com/<gh-user>/ai-resume-analyzer.git
cd ai-resume-analyzer
git checkout -b feature-resume-upload
echo "Resume Upload Module" >> project.txt
git add .
git commit -m "Added resume upload module"
git push origin feature-resume-upload
git checkout main
git merge feature-resume-upload
git push origin main
```

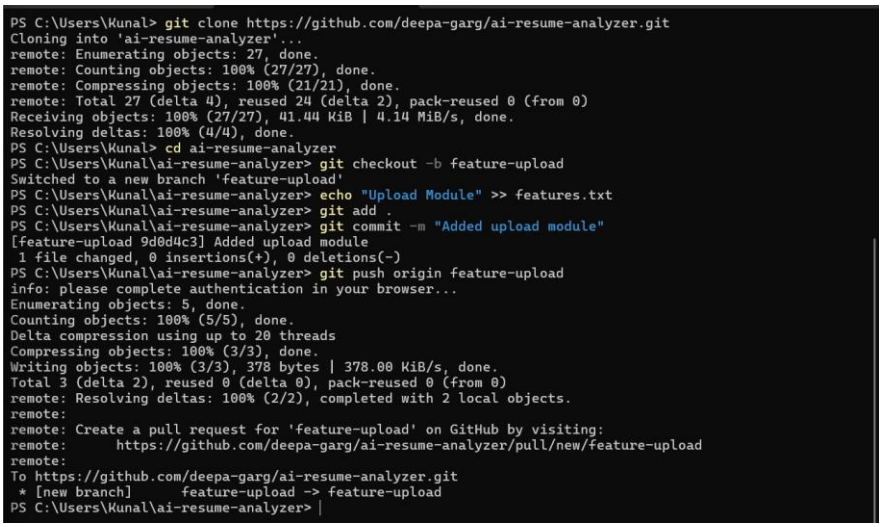


Figure 5.4: Cloning the repository, creating the feature branch, committing and pushing it (Steps 14–18)

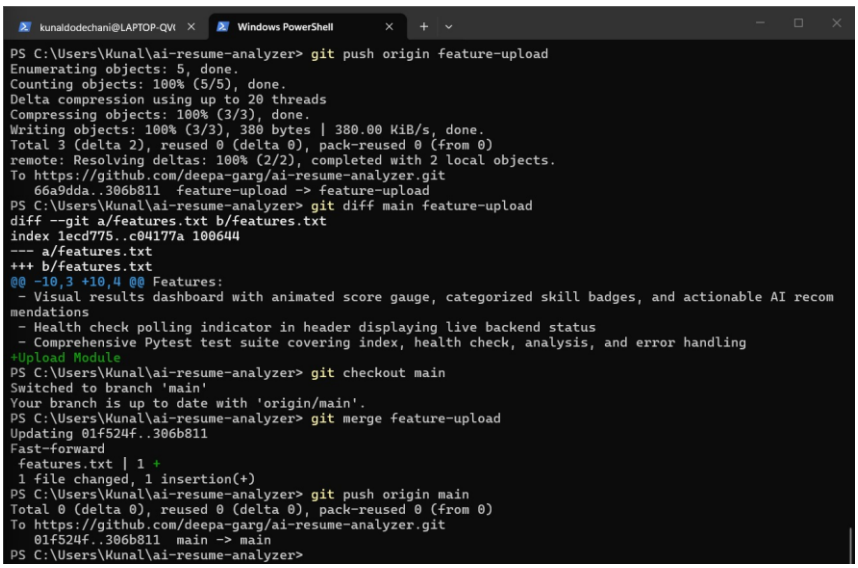


Figure 5.5: Comparing the branches, switching to main, merging the feature branch and pushing main

Steps 20 to 30

```
git checkout -b feature-score-dashboard
echo "Score Dashboard Module" >> project.txt
git add .
git commit -m "Added score dashboard module"
git push origin feature-score-dashboard
git checkout main
git pull origin main
git merge feature-score-dashboard
# CONFLICT in project.txt: remove the three marker lines and keep both module lines
git add project.txt
git commit -m "Resolved merge conflict"
git push origin main
```

Mohit Gabra created the score dashboard branch from main before Kunal Dodechani merged, which is what makes both branches change the same part of project.txt. On Windows, the echo command of PowerShell 5.1 saves files as UTF-16, which Git treats as binary; the echo steps were therefore run in Command Prompt or Git Bash.

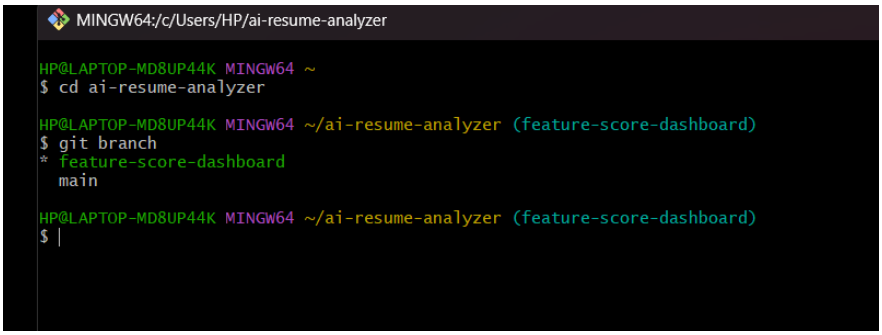


Figure 5.6: Score dashboard branch created and active (Step 20)

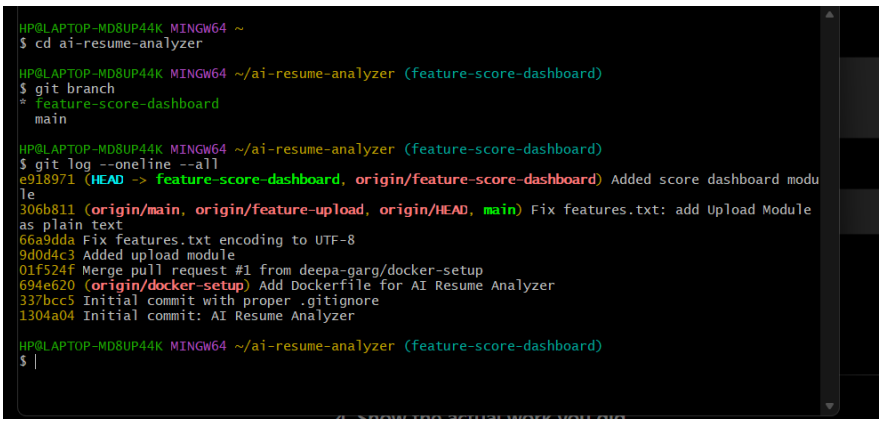


Figure 5.7: Commit history and project file after the dashboard module was added (Step 21)

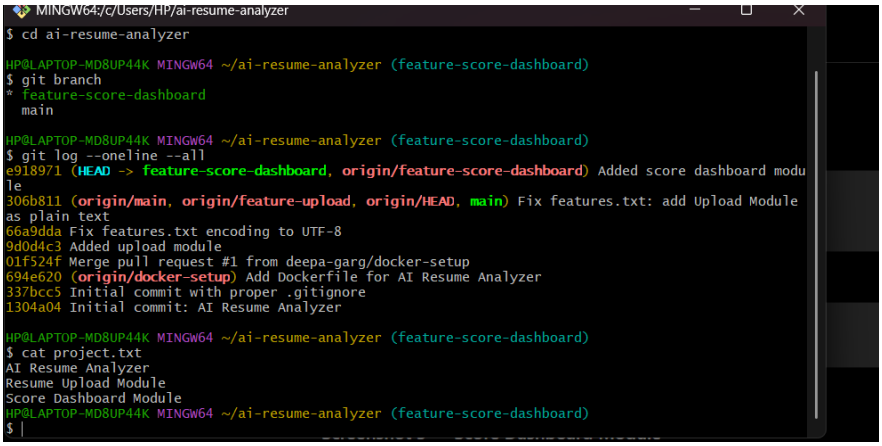


Figure 5.8: Committing the score dashboard feature (Step 23)

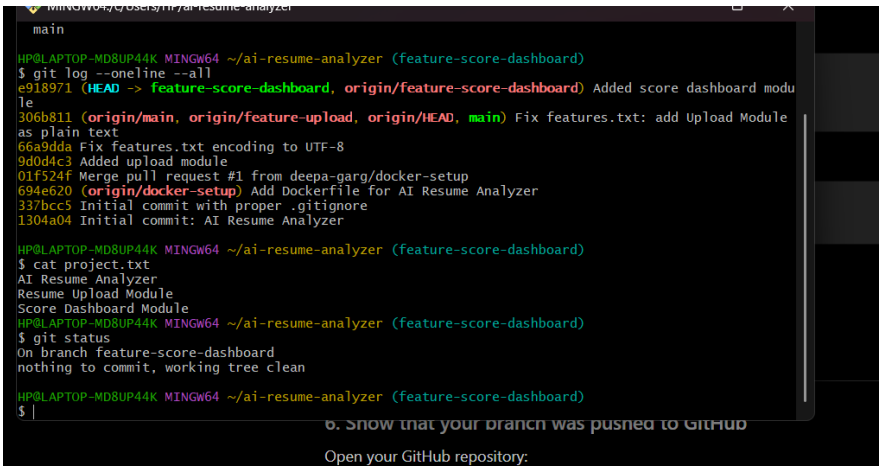


Figure 5.9: Pushing the score dashboard branch and checking the status (Step 24)

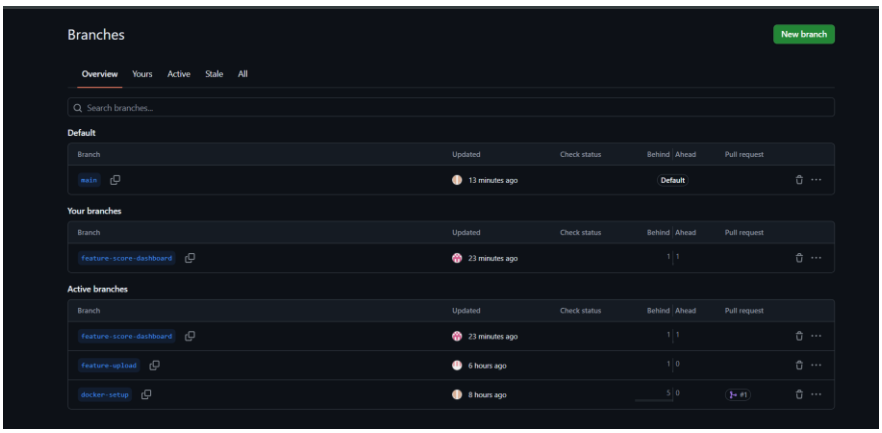


Figure 5.10: Branches overview on GitHub showing main and the feature branches

History, rollback, revert and restore: Steps 31 to 35

```
git pull origin main
git fetch --all
git log --oneline --graph --all          # Deepa Garg, Step 31
git branch -a                            # Deepa Garg, Step 32
git checkout -b practice-reset           # Kunal Dodechani, Step 33
git reset --soft HEAD~1
git checkout -b practice-revert          # Mohit Gabra, Step 34
git revert <commit-id>
git restore .                            # Deepa Garg, Step 35
```

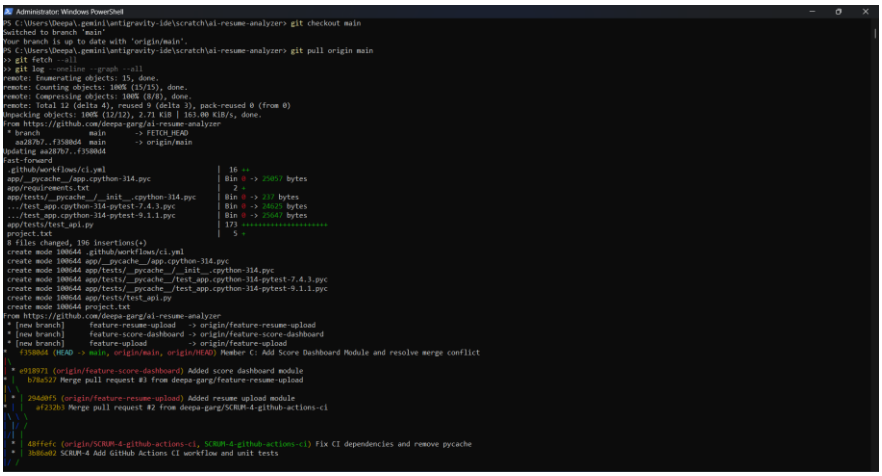


Figure 5.11: git pull and git fetch --all before viewing the history (Step 31)

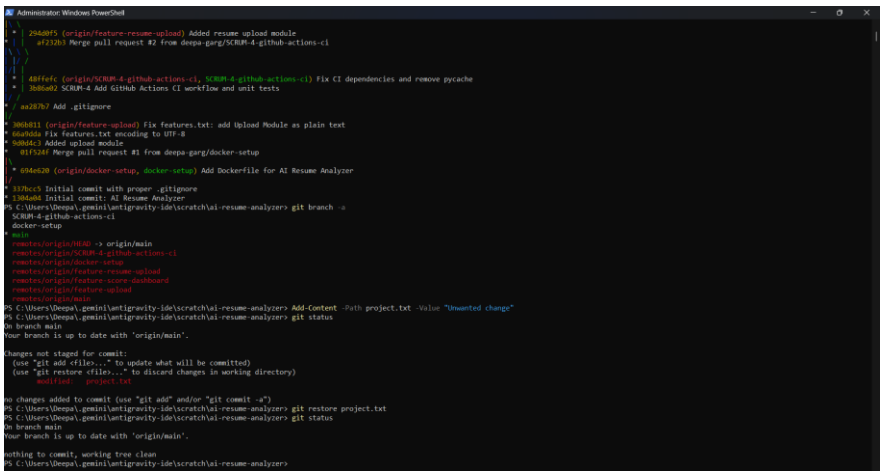


Figure 5.12: Commit graph, branch list and restore of uncommitted changes (Steps 31, 32 and 35)

5.1.4 Conclusion

A Git repository was created and connected to GitHub, feature branches were created and merged, a merge conflict was resolved, and rollback, revert, restore and history operations were performed.

5.2 Containerisation with Docker (Experiment 2)

5.2.1 Aim

To containerise the application using Docker **(LO4: containerize, orchestrate and deploy applications using Docker, Docker Compose, Kubernetes and cloud infrastructure tools).** The AI Resume Analyzer is packaged as a Docker image and run as a container, and the 25 Docker commands of the lab file are performed, each member running them on their own PC.

5.2.2 Theory

A container packages an application with its libraries and settings and runs it as an isolated process that shares the host kernel, so it starts in seconds and behaves the same on every machine. An image is a read-only, layered blueprint described by a Dockerfile and identified by a name and tag (for example resume-analyzer:v1); a container is a running instance of an image. On Windows, Docker Desktop runs the engine on the WSL 2 backend, which needs hardware virtualisation to be enabled in the BIOS.

5.2.3 Implementation

Installation (all members)

1. Enable Intel VT-x or AMD-V in the BIOS/UEFI settings (check in Task Manager, Performance, CPU, Virtualization: Enabled).
2. Install WSL 2 with wsl --install in PowerShell as Administrator and restart the computer.
3. Download and install Docker Desktop with the default settings (WSL 2 recommended).

4. Launch Docker Desktop and wait until the engine shows as running.
5. Verify the installation with docker --version and docker run hello-world.

Dockerfile, image build and container

Deepa Garg wrote the Dockerfile and .dockerignore, committed them on the branch docker-setup and merged it into main through a pull request. Kunal Dodechani and Mohit Gabra then pulled main and built the image on their own PC.

```
FROM python:3.11-slim
WORKDIR /app
COPY app/requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app/ .
EXPOSE 5000
CMD ["python", "app.py"]
```

```
git checkout -b docker-setup
git add Dockerfile .dockerignore
git commit -m "Add Dockerfile for AI Resume Analyzer"
git push origin docker-setup

docker build -t resume-analyzer:v1 .
docker run -d --name resume-app -p 5000:5000 resume-analyzer:v1
```

The containerised application is confirmed by opening http://localhost:5000/health.

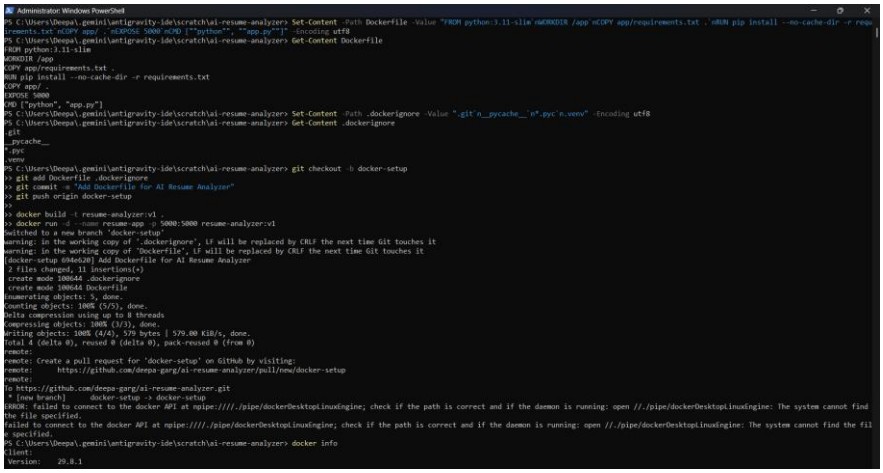


Figure 5.13: Dockerfile and .dockerignore created, committed on docker-setup and pushed

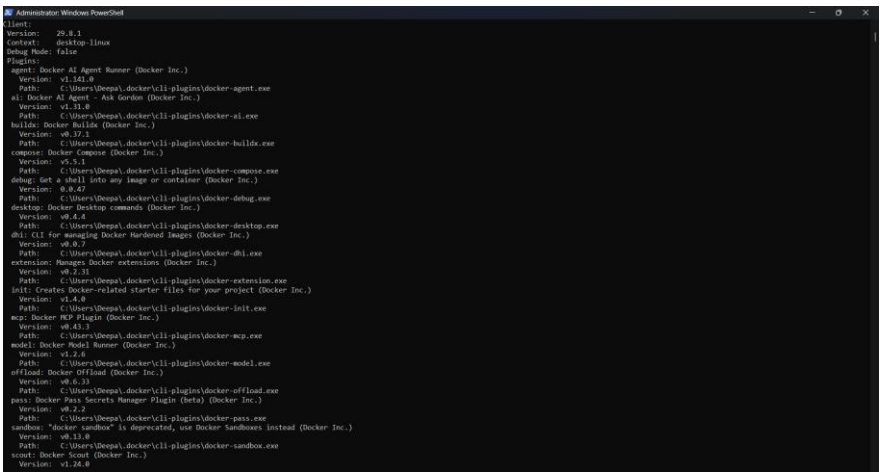


Figure 5.14: docker info: client information and installed plugins

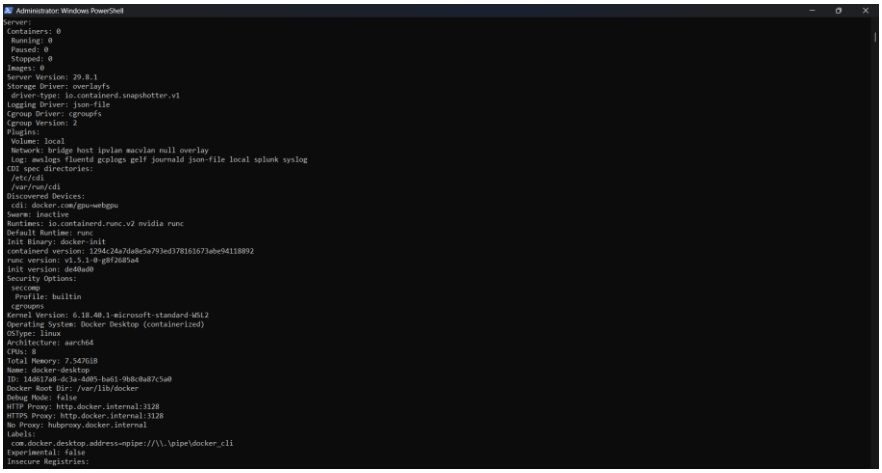


Figure 5.15: docker info: server information

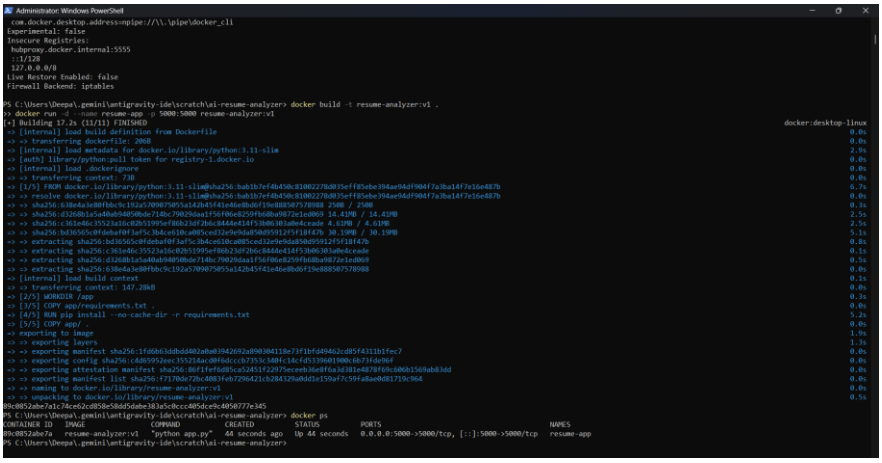


Figure 5.16: docker build of resume-analyzer:v1 and docker run of the resume-app container

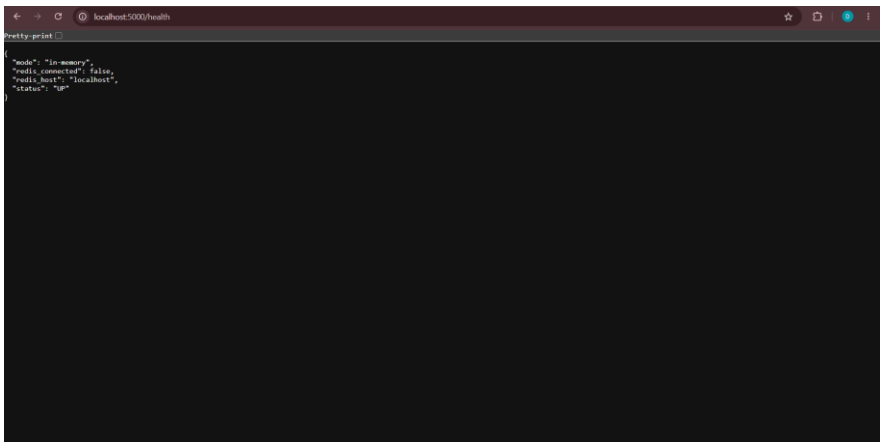


Figure 5.17: Containerised application responding at /health

Docker commands 1 to 25

No.	Command	Purpose
1	docker --version	Check the Docker version
2	docker search ubuntu --filter stars=500	Search Docker Hub with a filter
3	docker pull ubuntu	Download an image
4	docker run -it ubuntu /bin/bash	Run an interactive container
5	docker ps	List running containers
6	docker run --name mycontainer -it ubuntu bash	Create and start a named container
7	docker stop mycontainer	Stop a container gracefully
8	docker start mycontainer	Restart a stopped container
9	docker inspect mycontainer	Show detailed information about an object
10	docker ps -a	Show running and stopped containers
11	docker stats	Real-time resource usage
12	docker rm -f mycontainer	Remove a container (forced)
13	docker run -d --name web nginx	Run nginx in the background
14	docker pull alpine	Download the Alpine Linux image
15	docker search python	Search for an image
16	docker image prune	Remove dangling images
17	docker container ls	List running containers
18	docker info	System-wide information

No.	Command	Purpose
19	docker history <image>	Layers of an image
20	docker system df	Disk space used by Docker
21	docker network inspect bridge	Details of a network
22	docker volume create resume-data	Create persistent storage
23	docker network create resume-net	Create a network
24	docker system prune	Remove unused objects
25	docker rename resume-app resume-analyzer-app	Rename a container

Table 5.2: The 25 Docker commands and their purpose

Commands 1 to 9 (Deepa Garg)

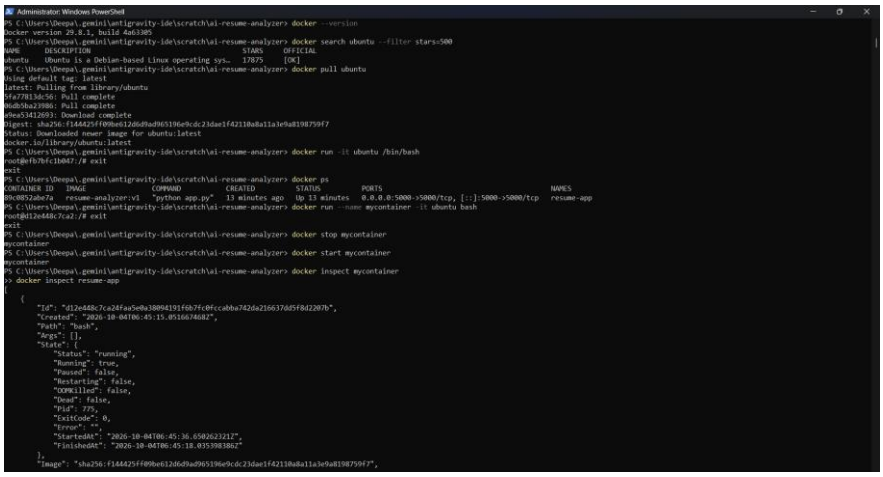


Figure 5.18: Version, search, pull, run, ps, stop, start and inspect

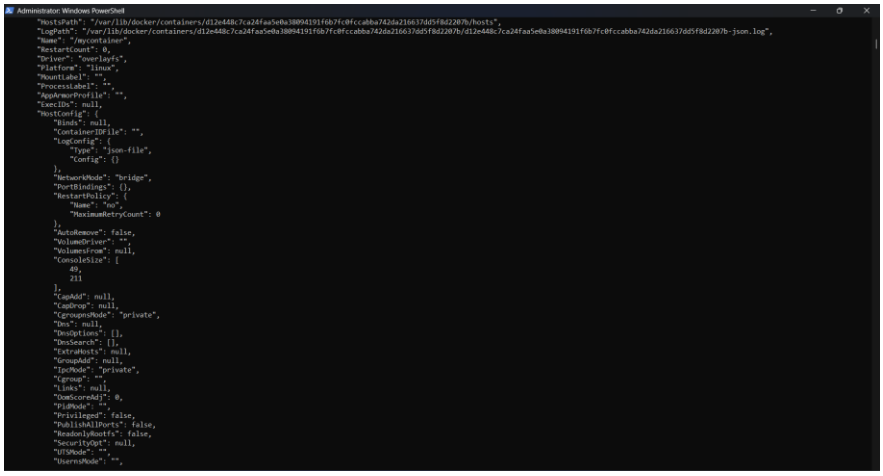


Figure 5.19: docker inspect output (continued)

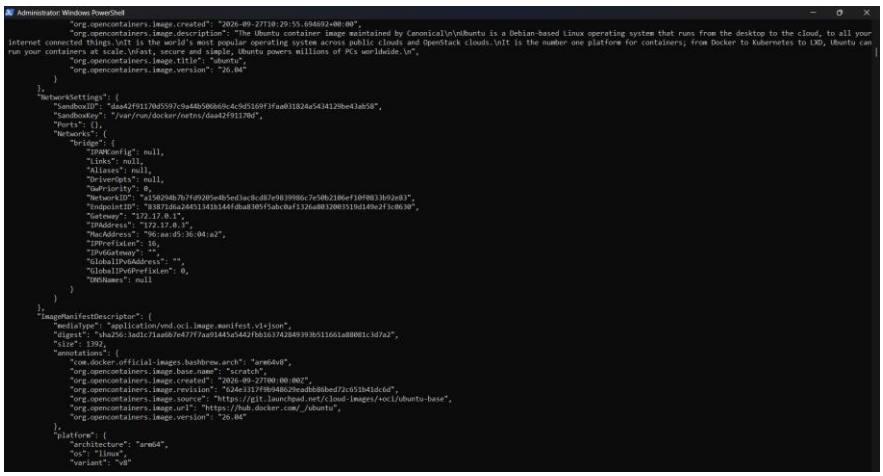


Figure 5.20: docker inspect output: image details and network settings

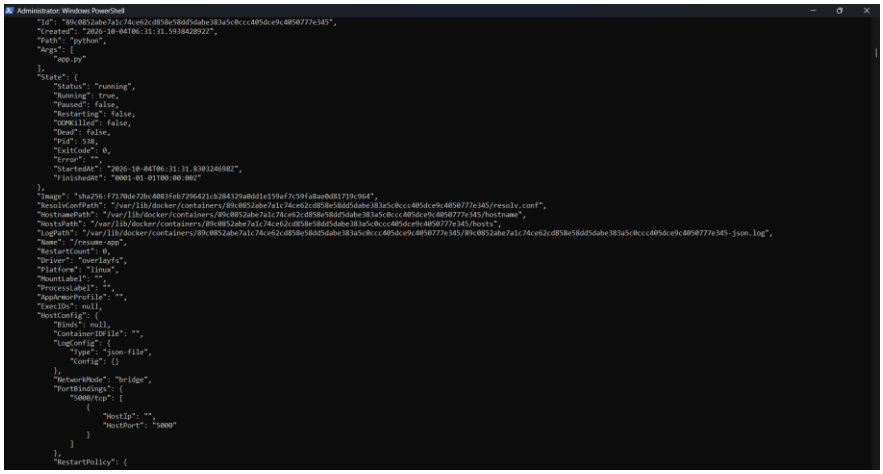


Figure 5.21: docker inspect resume-app: container state and configuration

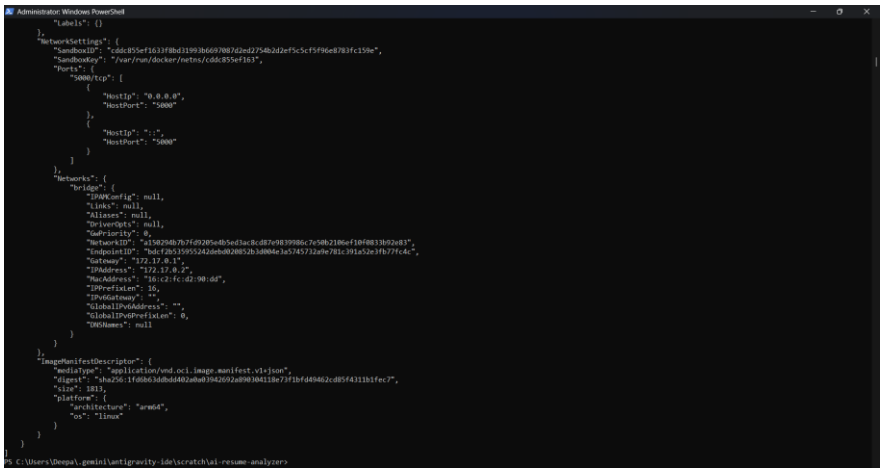


Figure 5.22: docker inspect resume-app: port bindings and network settings

Image build and commands 10 to 17 (Kunal Dodechani)

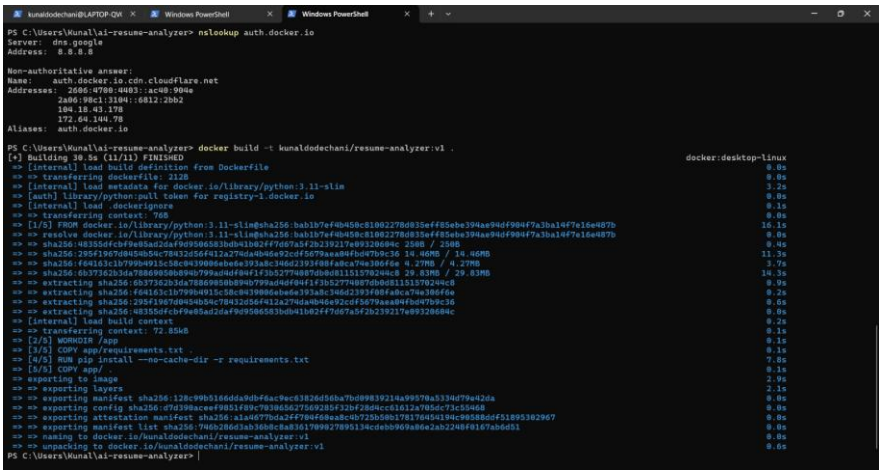


Figure 5.23: Building the image resume-analyzer:v1 on Kunal Dodechani's PC

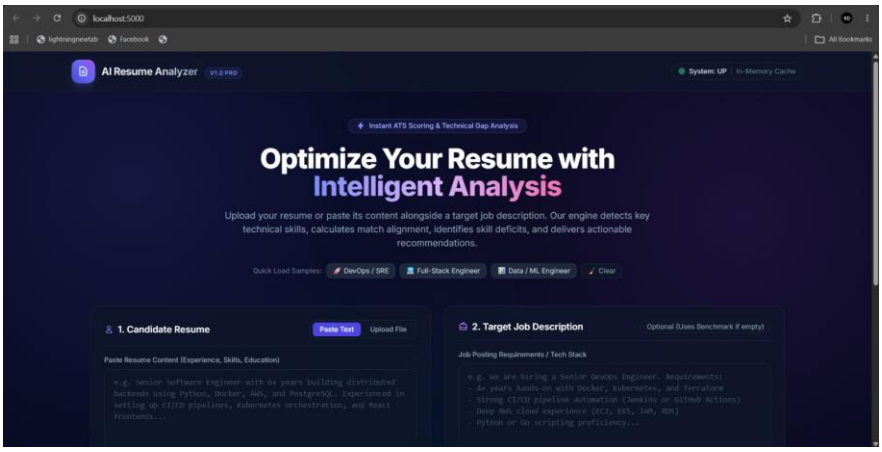


Figure 5.24: Application running at localhost:5000



Figure 5.25: Health endpoint of the running container

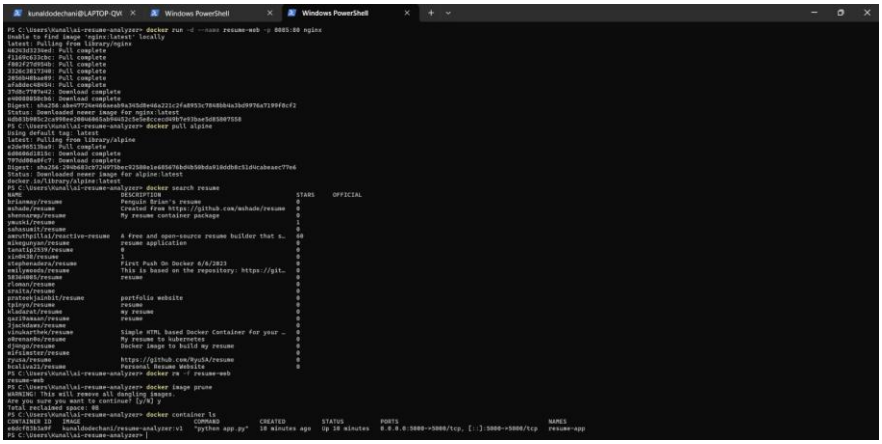


Figure 5.26: Running nginx, pulling alpine, searching python, pruning images and listing containers

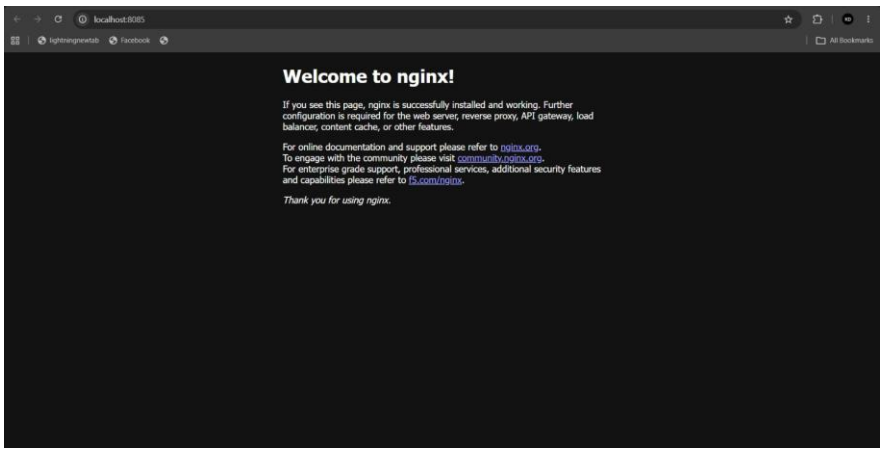


Figure 5.27: nginx container serving its welcome page

Installation and commands 18 to 25 (Mohit Gabra)

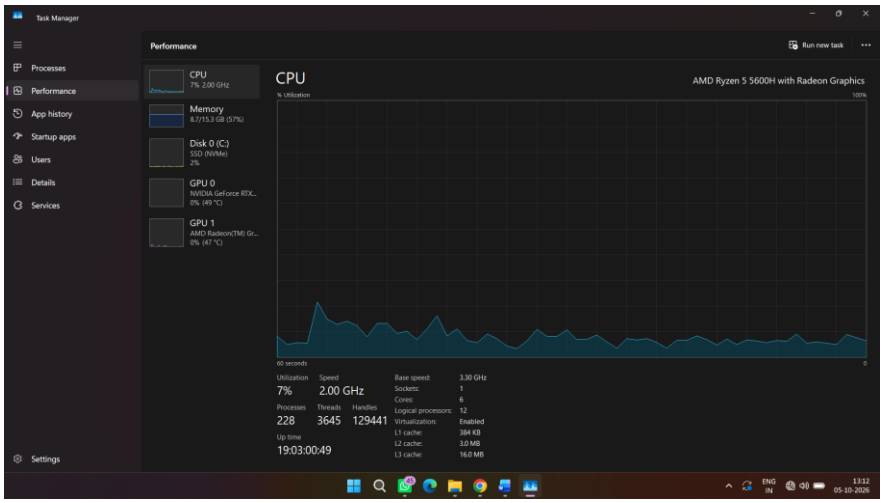


Figure 5.28: Task Manager showing virtualization enabled

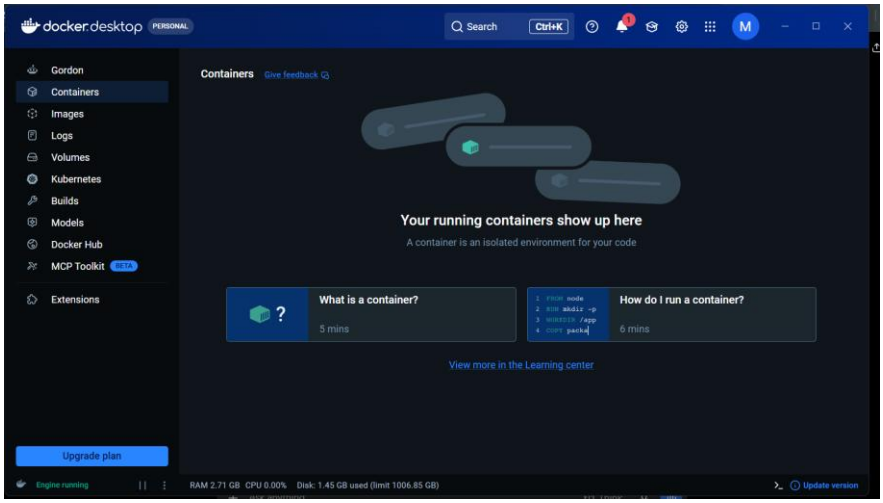


Figure 5.29: Docker Desktop running

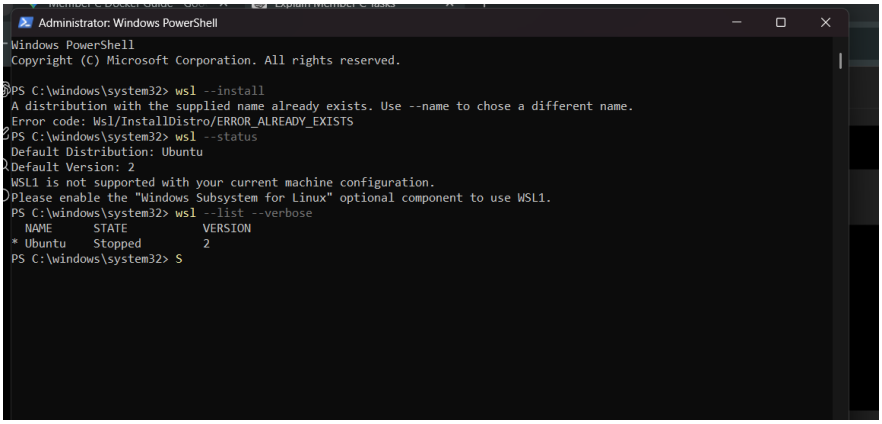


Figure 5.30: WSL 2 status

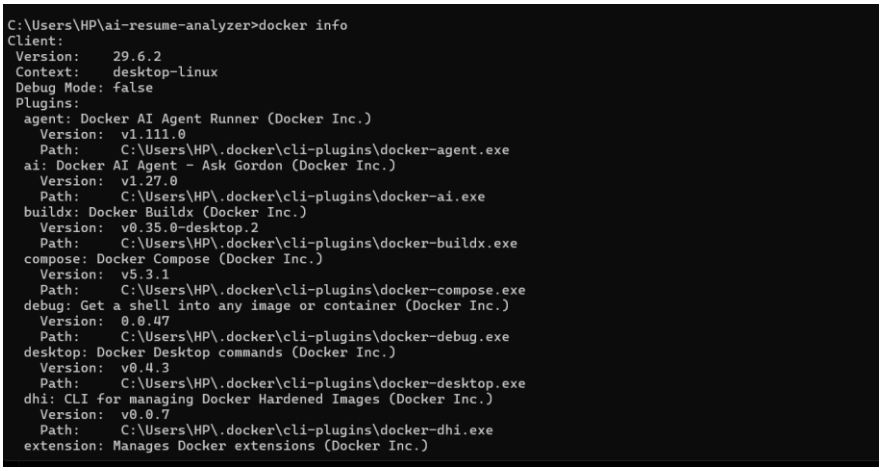


Figure 5.31: docker info (Command 18): client

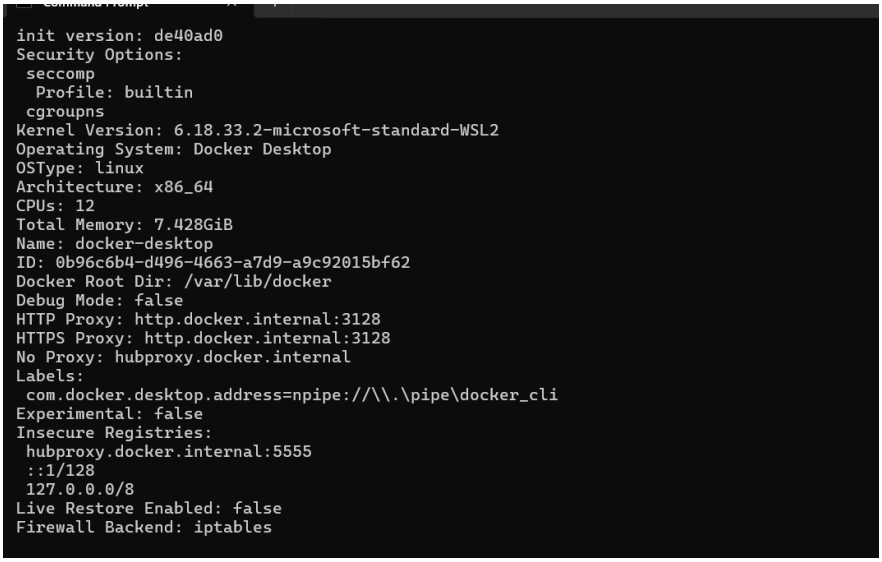


Figure 5.32: docker info (Command 18): server and kernel details

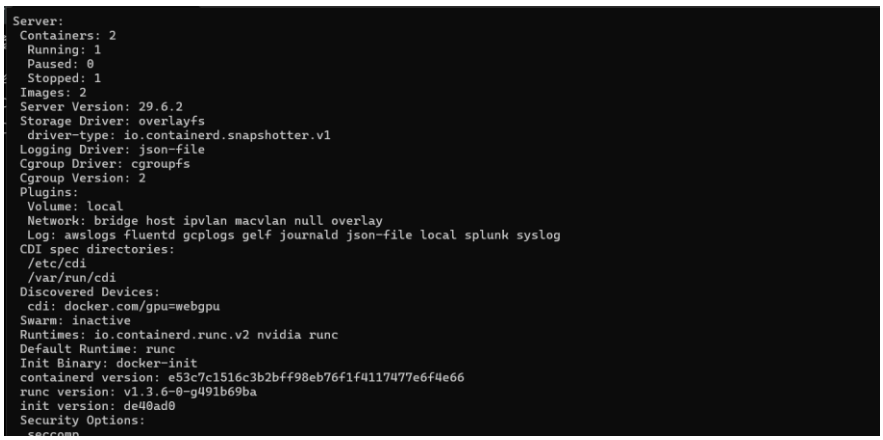


Figure 5.33: docker info (Command 18): storage and runtimes

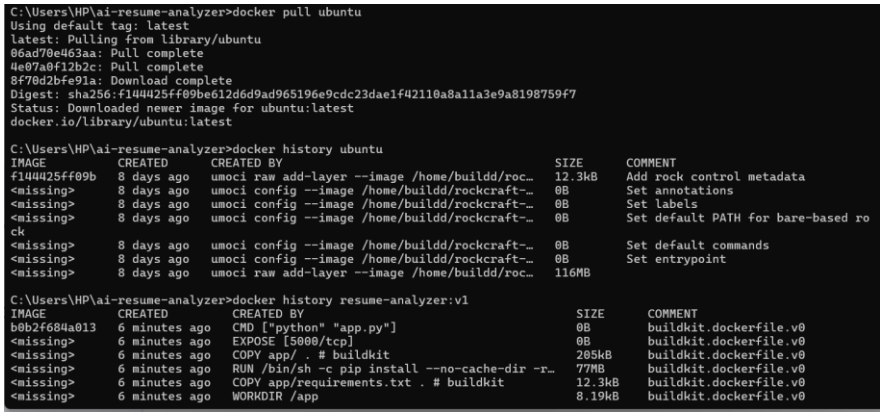


Figure 5.34: docker history of ubuntu and resume-analyzer:v1 (Command 19)

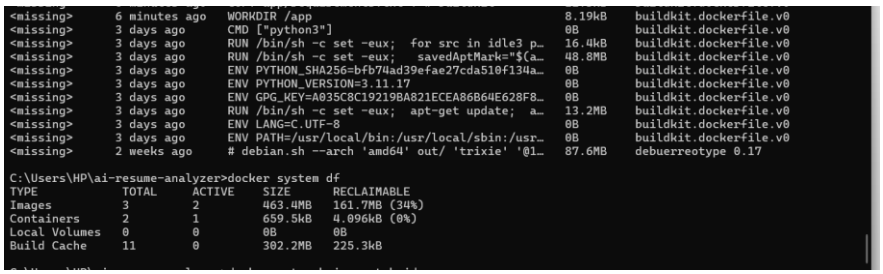


Figure 5.35: docker history output (continued) and docker system df

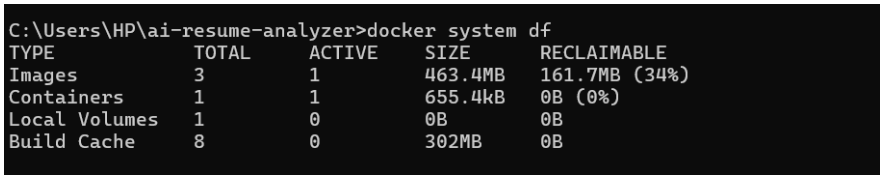


Figure 5.36: docker system df (Command 20)

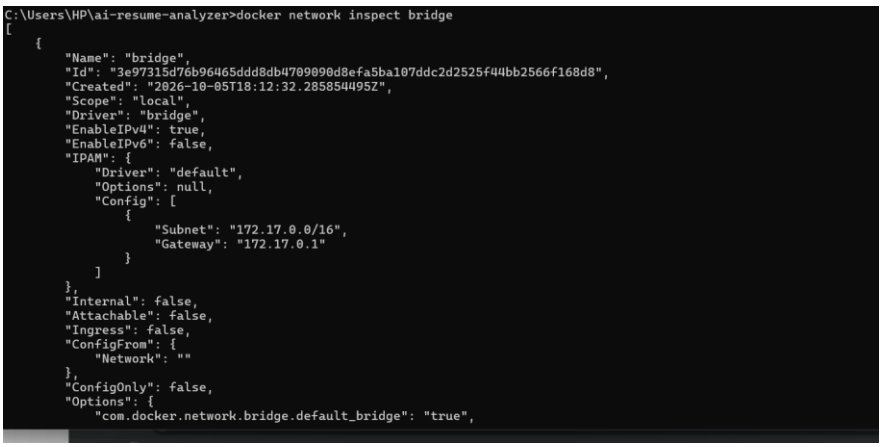


Figure 5.37: docker network inspect bridge (Command 21)

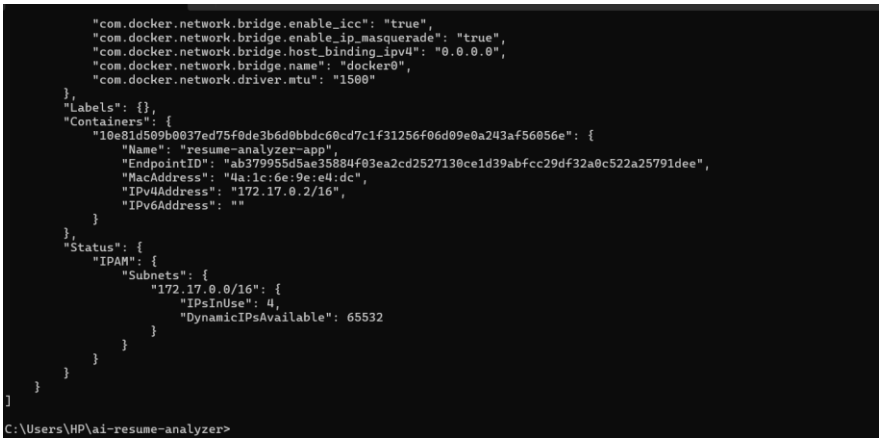


Figure 5.38: docker network inspect bridge (continued)



Figure 5.39: docker volume create (Command 22)



Figure 5.40: docker network create (Command 23)

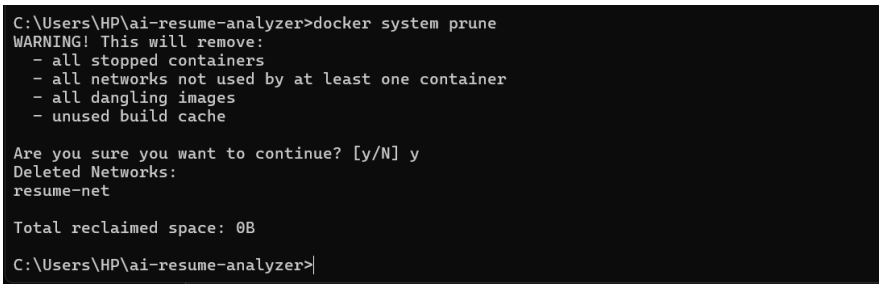


Figure 5.41: docker system prune (Command 24)

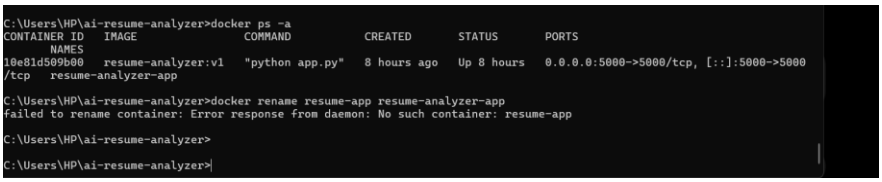


Figure 5.42: docker ps -a and docker rename (Command 25)

5.2.4 Conclusion

Docker packages the application with its dependencies into a container, giving a lightweight and consistent environment. The difference between an image (a read-only blueprint) and a container (a running instance) was demonstrated with the resume analyzer image and container.

5.3 Multi-service Deployment with Docker Compose (Experiment 3)

5.3.1 Aim

To deploy multi-service applications using Docker Compose. The application and its supporting services (nginx web server, MySQL database, Redis cache and phpMyAdmin) are defined in one docker-compose.yml file and started with one command.

5.3.2 Theory

Docker Compose describes a multi-container application in one YAML file and manages it as a unit. Each entry under services: defines a container with its image or build context, ports, environment variables, volumes and dependencies. Compose creates a project network on which services reach each other by service name (for example phpMyAdmin connects to the host database and nginx forwards to http://app:5000), and a volume keeps database data beyond the life of a container. Ports are mapped host:container, so a port conflict is solved by changing the left-hand number.

Compose command	Purpose
docker compose config	Validate and display the configuration
docker compose up -d	Create and start all services in the background
docker compose ps / logs / images	Show container status / logs / images used
docker compose stop / start / restart	Stop, start or restart containers without removing them
docker compose down	Stop and remove containers and networks
docker compose down -v	Also remove the volumes

Table 5.3: Docker Compose commands

5.3.3 Implementation

The three members built one docker-compose.yml together. Each added their services on their own branch (compose-a, compose-b, compose-c) and merged through a pull request in the order A, B, C. Containers left over from Experiment 2 were stopped first so that the ports were free.

Deepa Garg: services web and app

```
docker --version
mkdir compose-demo
cd compose-demo
```

```
services:
```

```
  web:
    image: nginx
    ports:
      - "8081:80"
    volumes:
      - ./nginx.conf:/etc/nginx/conf.d/default.conf:ro
  app:
    build:
      context: ..
      dockerfile: Dockerfile
    environment:
      REDIS_HOST: redis
```

```
server {
    listen 80;
    location / {
        proxy_pass http://app:5000;
    }
}
```

```
docker compose down
docker compose up -d
docker compose config
docker compose up -d
```

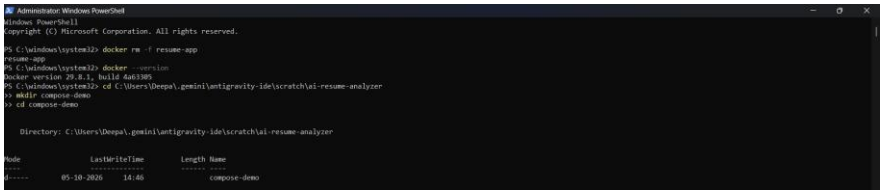


Figure 5.43: Docker version and creation of the compose-demo folder

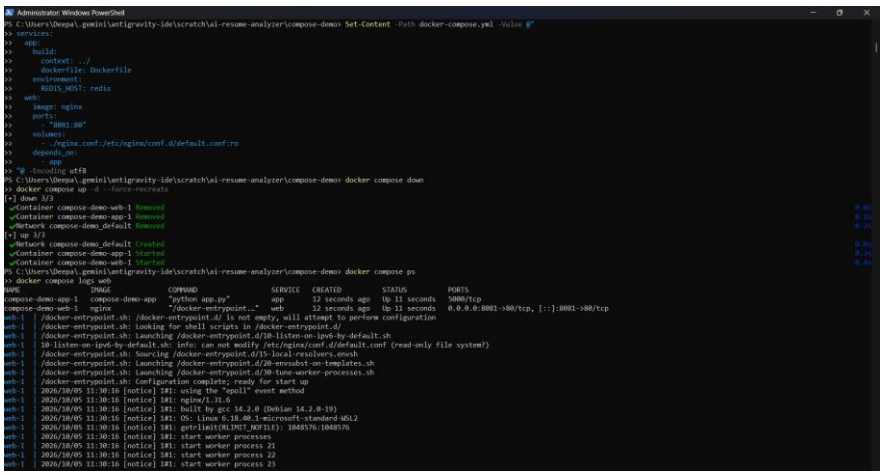


Figure 5.44: docker compose down, up -d and config for the web and app services

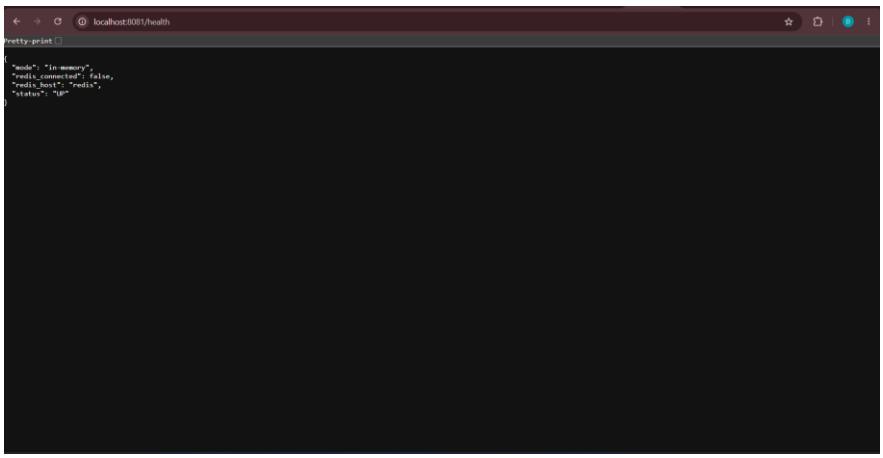


Figure 5.45: Opening the pull request for the compose-a branch on GitHub



Figure 5.46: Committing and pushing the compose-a branch

Kunal Dodechani: services database and phpmyadmin

```
database:
  image: mysql:8.0
  environment:
    MYSQL_ROOT_PASSWORD: root123
phpmyadmin:
  image: phpmyadmin
  ports:
    - "8082:80"
  environment:
    PMA_HOST: database
```

```
docker compose config
docker compose up -d
docker compose ps
docker compose logs
docker compose images
```

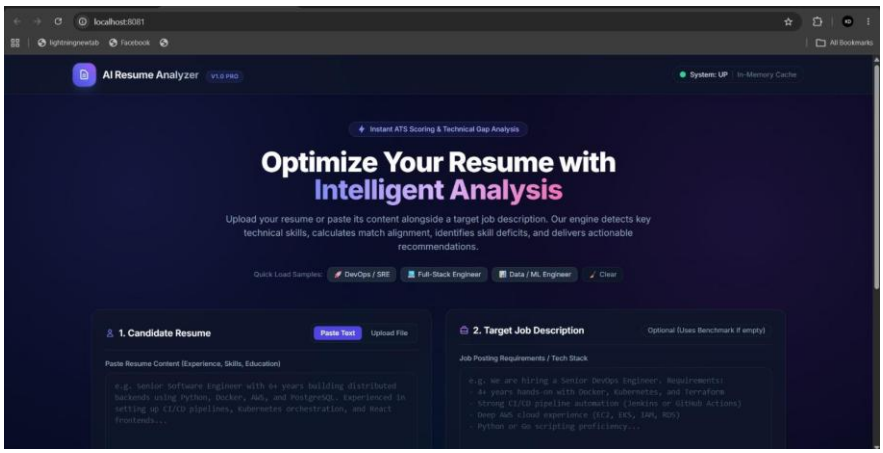


Figure 5.47: Application reachable through nginx at http://localhost:8081

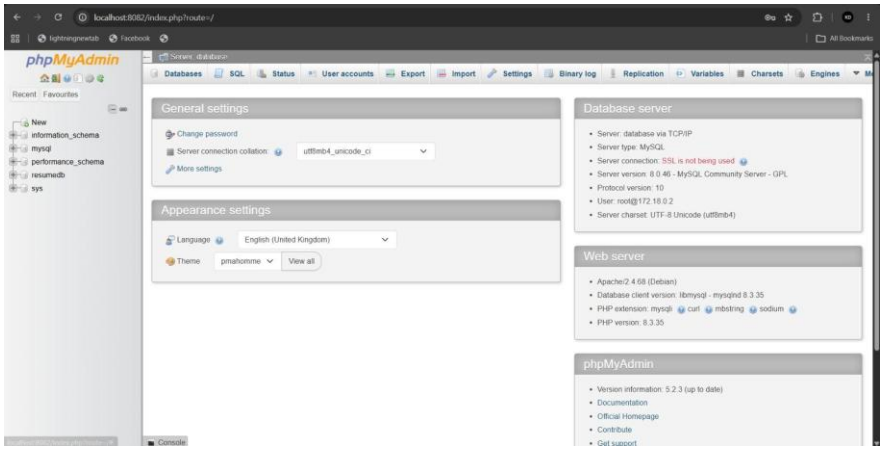


Figure 5.48: phpMyAdmin at http://localhost:8082



Figure 5.49: docker compose ps showing the running services

```
PS C:\Users\HP> cd "C:\Users\HP\ai-resume-analyzer\compose-demo"
PS C:\Users\HP\ai-resume-analyzer\compose-demo> docker compose config
name: compose-demo
services:
  app:
    build:
      context: C:\Users\HP\ai-resume-analyzer
      dockerfile: Dockerfile
    environment:
      REDIS_HOST: redis
    networks:
      default: null
  database:
    environment:
      MYSQL_DATABASE: resumedb
      MYSQL_ROOT_PASSWORD: root123
    image: mysql:8.0
    networks:
      default: null
    volumes:
      - type: volume
        source: db_data
        target: /var/lib/mysql
        volume: {}
  phpmyadmin:
    environment:
      PMA_HOST: database
    image: phpmyadmin
    networks:
      default: null
```

Figure 5.50: docker compose config with the database and phpmyadmin services

Mohit Gabra: service redis and lifecycle commands

redis:

```
image: redis
```

```
docker compose config
docker compose up -d
docker compose ps
docker compose stop
docker compose start
docker compose restart
docker compose down
docker compose up -d
docker compose down -v
docker compose config
```

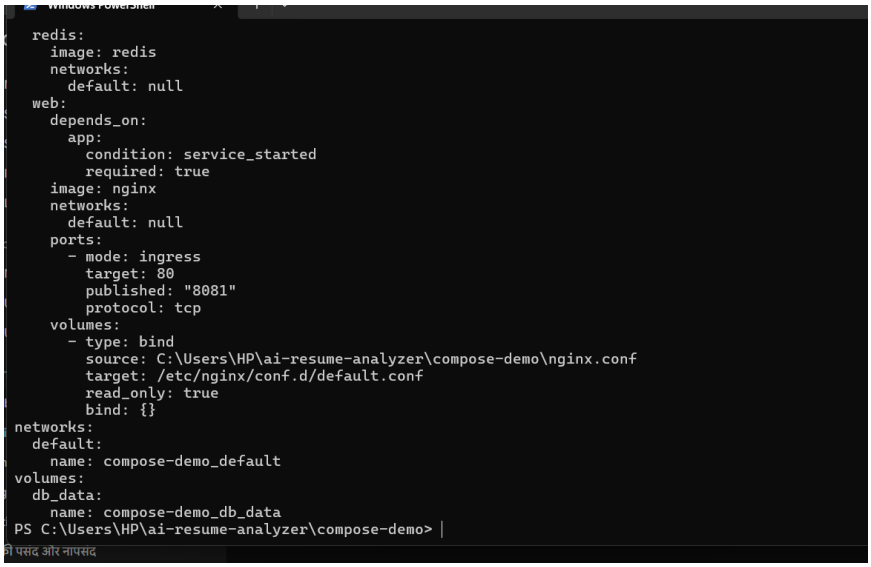


Figure 5.51: docker compose config (continued): redis, web, network and volume

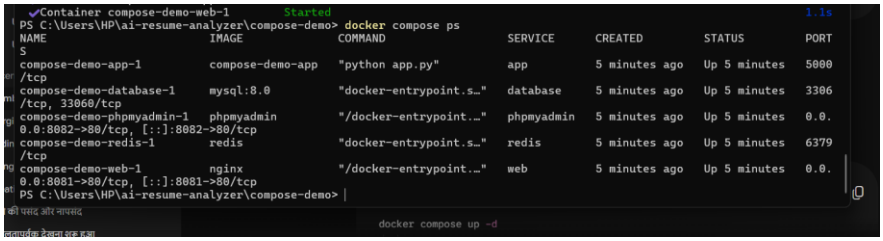


Figure 5.52: docker compose ps: all five services running

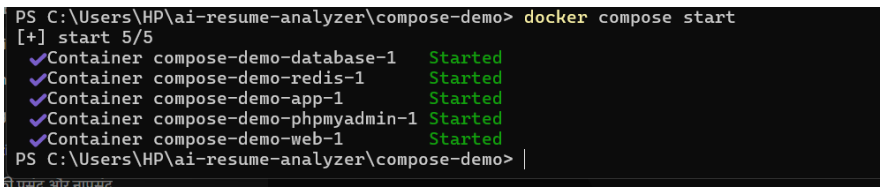


Figure 5.53: docker compose start

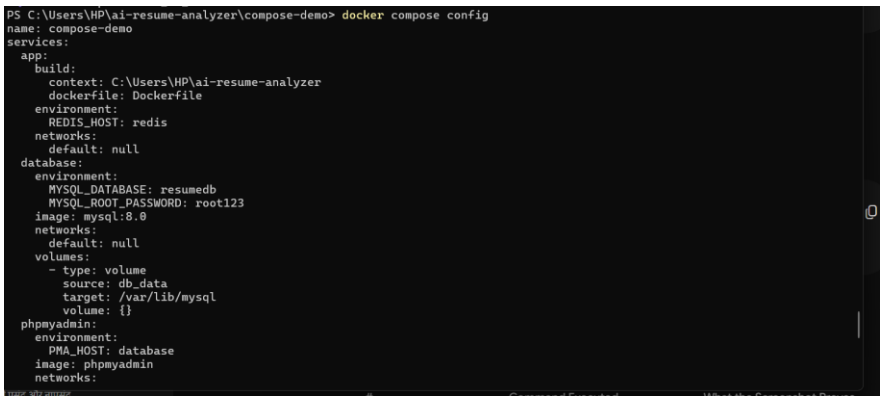


Figure 5.54: docker compose config of the complete file

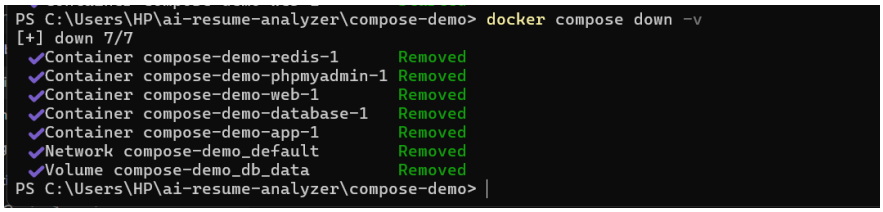


Figure 5.55: docker compose down -v removing containers, network and volume

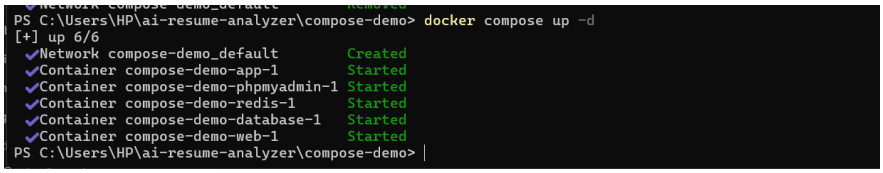


Figure 5.56: docker compose up -d

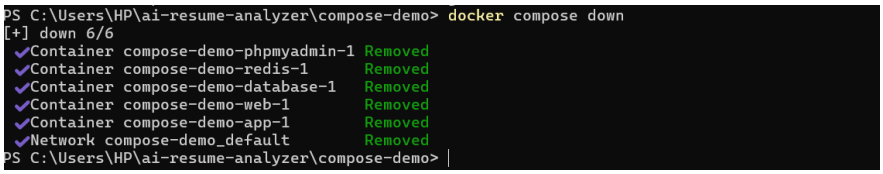


Figure 5.57: docker compose down

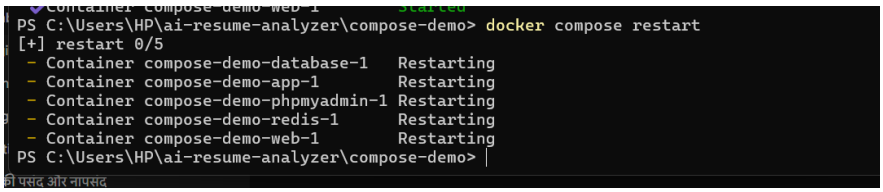


Figure 5.58: docker compose restart

5.3.4 Conclusion

Docker Compose created and managed a multi-container application from one YAML file. Services, images, environment variables and port mapping were used, and the essential Compose commands were performed, including resolving port conflicts.

5.4 Continuous Integration with Jenkins (Experiment 4)

5.4.1 Aim

To understand Continuous Integration, install Jenkins, and build CI/CD pipelines using Declarative and Scripted syntax (LO3). A Jenkins server is installed and two pipeline jobs, one Declarative and one Scripted, run Build, Test and Deploy stages for the AI Resume Analyzer.

5.4.2 Theory

Continuous Integration (CI) merges every developer's changes into the shared branch frequently and verifies each merge with an automatic build and tests; Continuous Delivery / Deployment (CD) extends this by preparing and releasing every tested build automatically. Jenkins is an open-source automation server written in Java, started here with java -jar jenkins.war on port 8080 and extended by plugins. A Pipeline job expresses the build as code, divided into stages (Build, Test and Deploy) that contain steps such as echo.

Aspect	Declarative Pipeline	Scripted Pipeline
Outer structure	pipeline { agent, stages, stage, steps }	node { stage(...) { ... } }
Syntax	Structured and predefined, easy to read and validate	Free-form Groovy, more flexible
Error handling	Built-in post sections	Written by hand with try / catch
Typical use	Most projects and the Jenkinsfile stored in source control	Complex logic that needs full programming control

Table 5.4: Declarative and Scripted pipelines compared

5.4.3 Implementation

Procedure steps 1 to 3

```
java --version
cd C:\jenkins
java -jar jenkins.war
```

Jenkins was opened at http://localhost:8080 and unlocked with the initial administrator password (printed in the PowerShell window and stored in C:\Users\<you>\.jenkins\secrets\initialAdminPassword). Install suggested plugins was selected, the administrator account was created and the default Jenkins URL was kept. To let Kunal Dodechani and Mohit Gabra open Jenkins from their own PCs, the firewall rule below was added (PowerShell as Administrator) and one user was created for each of them under Manage Jenkins, Users, Create User.

```
ipconfig
New-NetFirewallRule -DisplayName "Jenkins 8080" -Direction Inbound -LocalPort
8080 -Protocol TCP -Action Allow
```

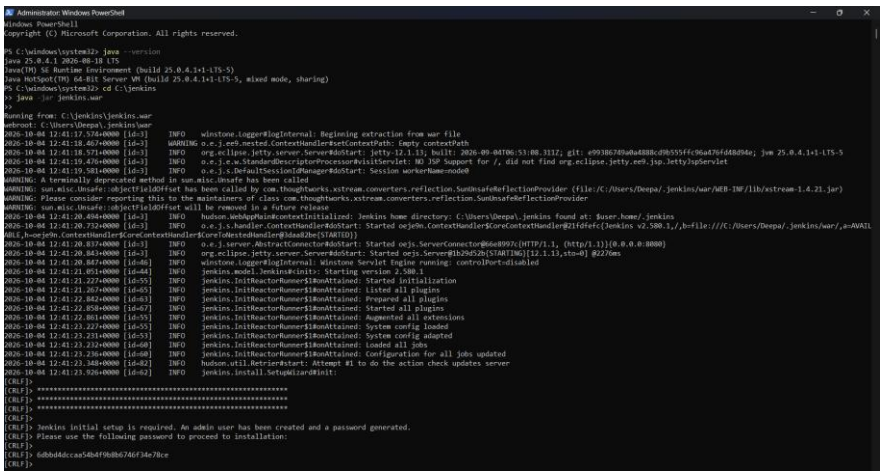


Figure 5.59: java --version and launch of jenkins.war with the initial administrator password

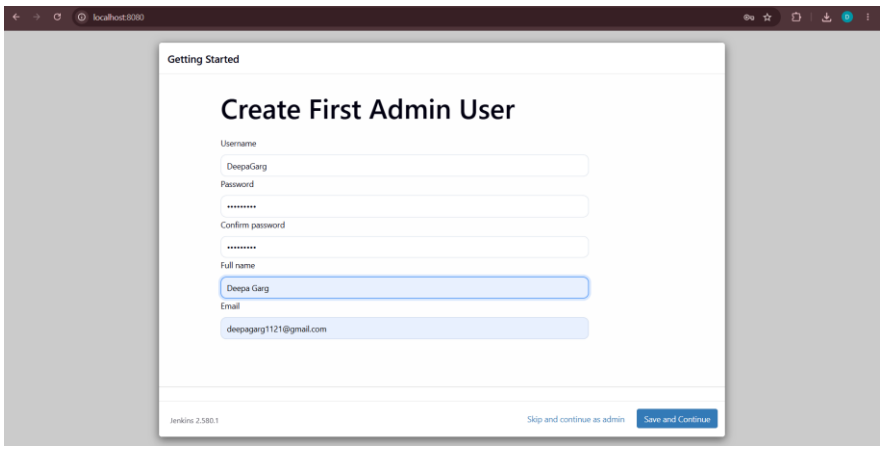


Figure 5.60: Creating the first administrator user

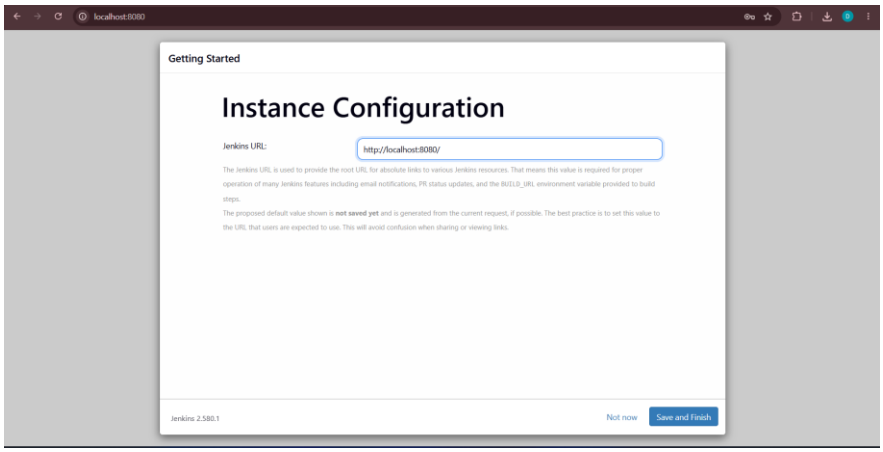


Figure 5.61: Jenkins instance configuration (Jenkins URL)

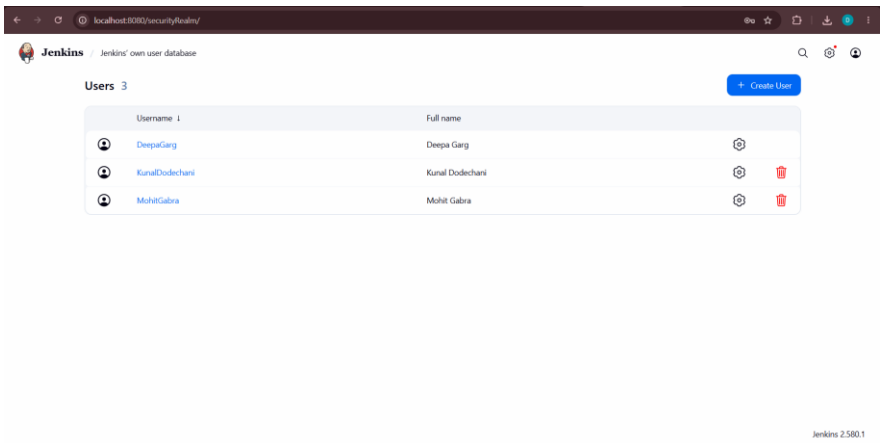


Figure 5.62: Jenkins user accounts for the three members

Declarative Pipeline (job ResumeAnalyzer-Declarative)

```
pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                echo 'Building the AI Resume Analyzer...'
            }
        }
        stage('Test') {
            steps {
                echo 'Testing the AI Resume Analyzer...'
            }
        }
        stage('Deploy') {
            steps {
                echo 'Deploying the AI Resume Analyzer...'
            }
        }
    }
}
```

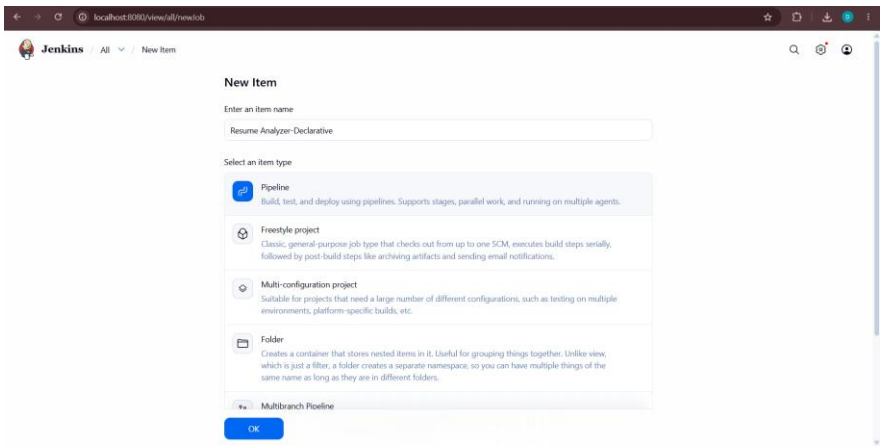


Figure 5.63: New Item screen for the job ResumeAnalyzer-Declarative

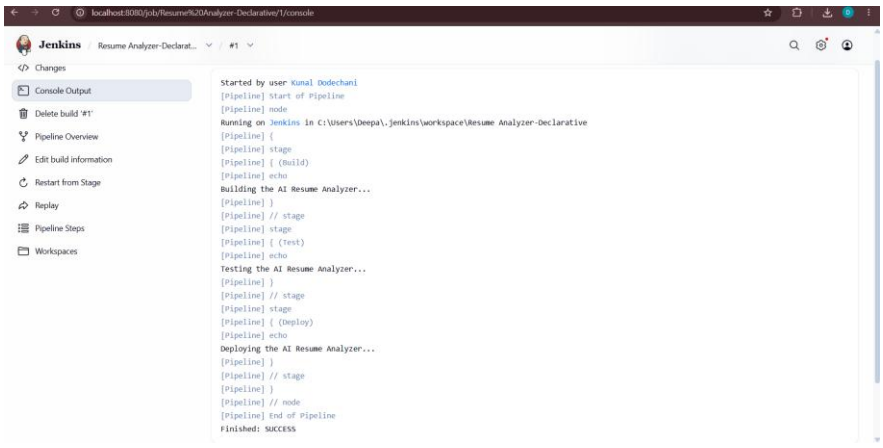


Figure 5.64: Console Output of the Declarative pipeline ending with Finished: SUCCESS

Scripted Pipeline (job ResumeAnalyzer-Scripted)

```
node {
    stage('Build') {
        echo 'Building the AI Resume Analyzer...'
    }
    stage('Test') {
        echo 'Testing the AI Resume Analyzer...'
    }
    stage('Deploy') {
        echo 'Deploying the AI Resume Analyzer...'
    }
}
```

In each case the job was created with New Item, Pipeline, the script was pasted in the Pipeline section and saved, and Build Now was clicked. Build #1 was opened and its Console Output checked for the status and the line Finished: SUCCESS.

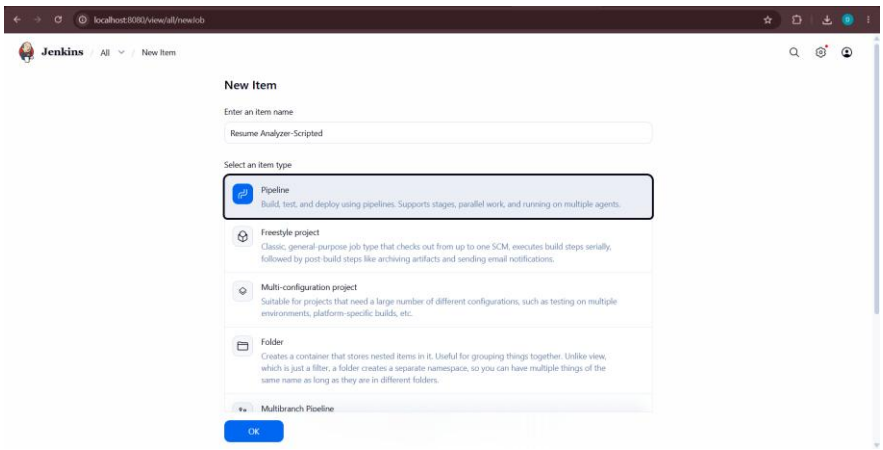


Figure 5.65: New Item screen for the job ResumeAnalyzer-Scripted

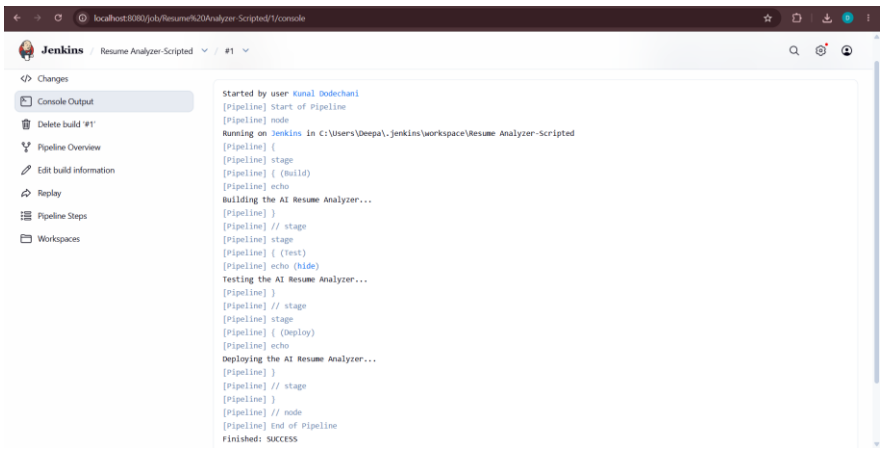


Figure 5.66: Console Output of the Scripted pipeline ending with Finished: SUCCESS

5.4.4 Conclusion

Jenkins was installed and configured, and both Declarative and Scripted Pipelines were created and executed, showing how Pipeline as Code automates build, test and deploy.

5.5 Automated Deployment with GitHub and Jenkins (Experiment 5)

5.5.1 Aim

To automate application deployment using GitHub and Jenkins (LO3). A Jenkinsfile with Build, Test and Deploy stages is stored in the GitHub repository, Jenkins fetches it with Pipeline script from SCM, and a push to GitHub triggers a build.

5.5.2 Theory

Keeping the pipeline in a Jenkinsfile in the same repository as the code (Pipeline as Code) versions, reviews and rolls it back together with the application; with Pipeline script from SCM, Jenkins clones the repository, checks out */main and reads the Jenkinsfile. To avoid polling, GitHub sends a webhook (here /github-webhook/) when code is pushed and the option GitHub hook trigger for GITScm polling starts the job; because Jenkins runs on a private PC, a public tunnel (ngrok) exposes port 8080. Build prepares the application, Test runs the automated tests and Deploy places the tested application in its target environment.

5.5.3 Implementation

Deepa Garg: Part 1 and Part 4

Java was checked with java --version, and Jenkins, already installed in Experiment 4, was confirmed to be running at http://localhost:8080. The Jenkinsfile was then added to the repository ai-resume-analyzer, in the root, with the commit message Add Jenkins pipeline for build, test, and deploy.

```
pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                echo 'Building the application...'
            }
```

```
        }
        stage('Test') {
            steps {
                echo 'Testing the application...'
            }
        }
        stage('Deploy') {
            steps {
                echo 'Deploying the application...'
            }
        }
    }
}
```

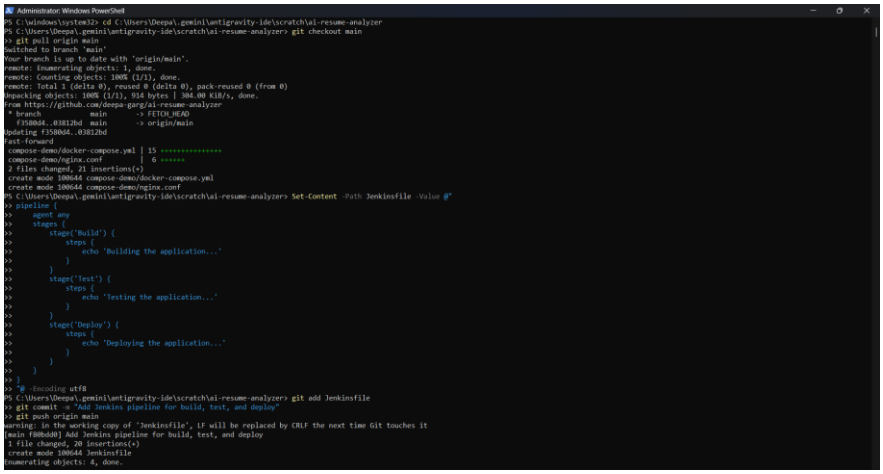


Figure 5.67: Creating the Jenkinsfile and committing it to the repository

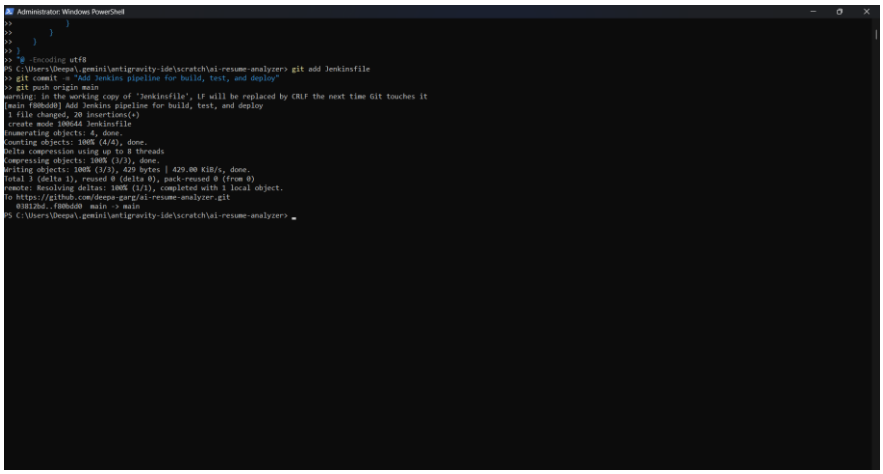


Figure 5.68: Pushing the Jenkinsfile to the main branch

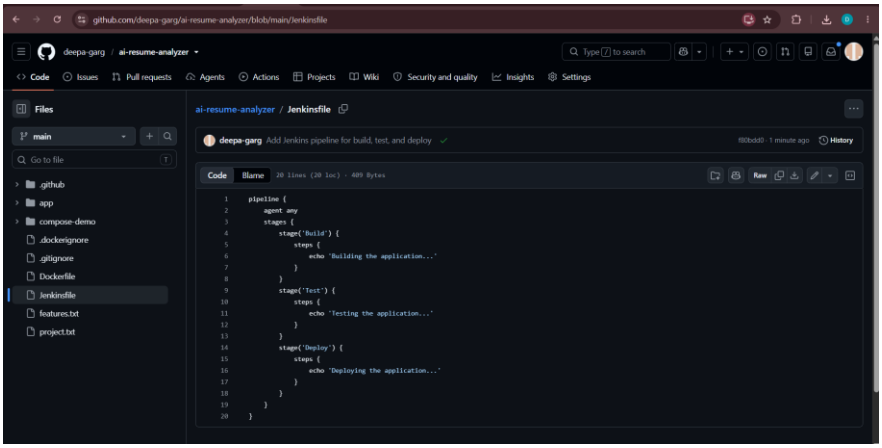


Figure 5.69: Jenkinsfile visible in the GitHub repository

Part 2, Part 3 and webhook

The job ResumeAnalyzer-Demo was created with New Item, Pipeline, the Declarative script was entered directly, and Build Now was clicked. The Console Output of build #1 shows the stages Build, Test and Deploy and ends with Finished: SUCCESS. For the webhook, ngrok was started on Deepa Garg's PC and a webhook was added in the GitHub repository settings.

```
ngrok http 8080

# GitHub: Settings, Webhooks, Add webhook
#   Payload URL : https://<ngrok-id>.ngrok-free.app/github-webhook/
#   Content type: application/json
#   Event       : Just the push event
```

Part 5 and Part 6

- Under Triggers, tick GitHub hook trigger for GITScm polling.
- Under Pipeline, change Definition from Pipeline script to Pipeline script from SCM.
- Set SCM to Git, enter the repository URL https://github.com/<gh-user>/ai-resume-analyzer.git, set Branch Specifier to */main and keep Script Path as Jenkinsfile.
- Save, click Build Now (or push a commit), and open the Console Output of build #2.

The console output of build #2 shows Jenkins obtaining the Jenkinsfile from GitHub, cloning it into the workspace, checking out the latest commit of main, running the Build, Test and Deploy stages and ending with Finished: SUCCESS.

Jenkins – Github Connectivity

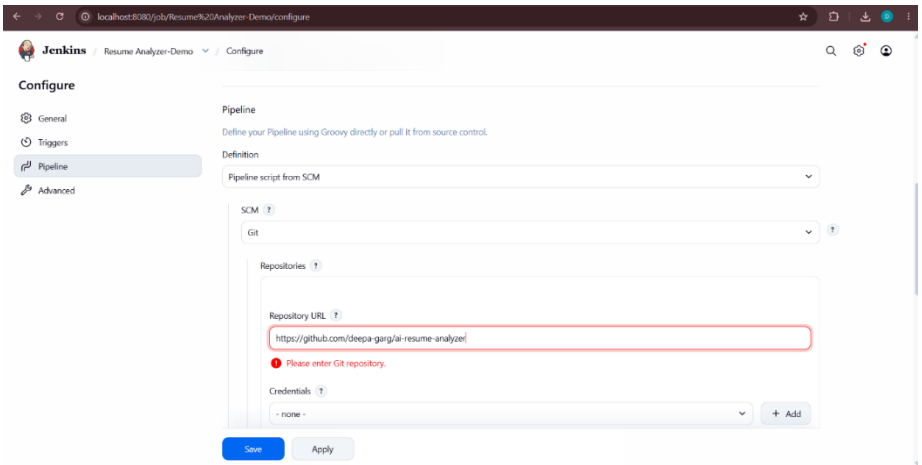


Figure 5.70: Job configuration: Pipeline script from SCM with Git selected and the repository URL entered

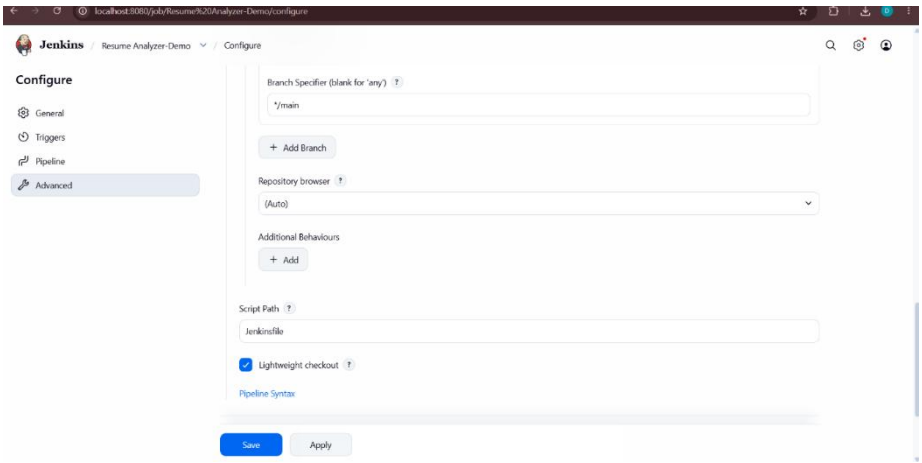


*Figure 5.71: Branch specifier */main and Script Path Jenkinsfile in the job configuration*

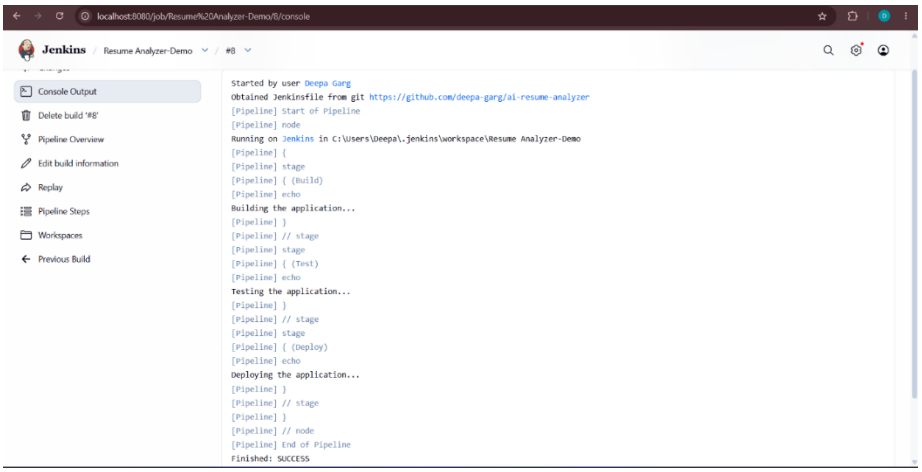


Figure 5.72: Console Output of the build that obtained the Jenkinsfile from GitHub, ending with Finished: SUCCESS

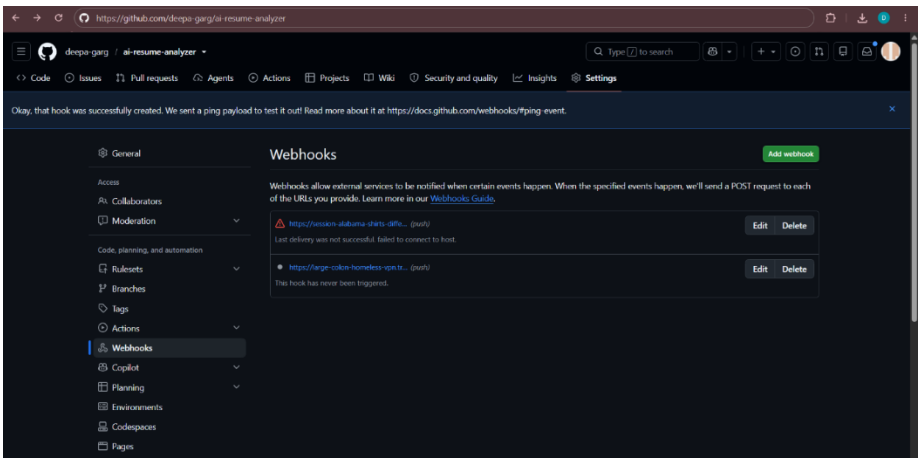


Figure 5.73: GitHub repository webhook configured for the Jenkins server (Settings, Webhooks)

Automatic trigger test: a change pushed to GitHub starts a Jenkins build

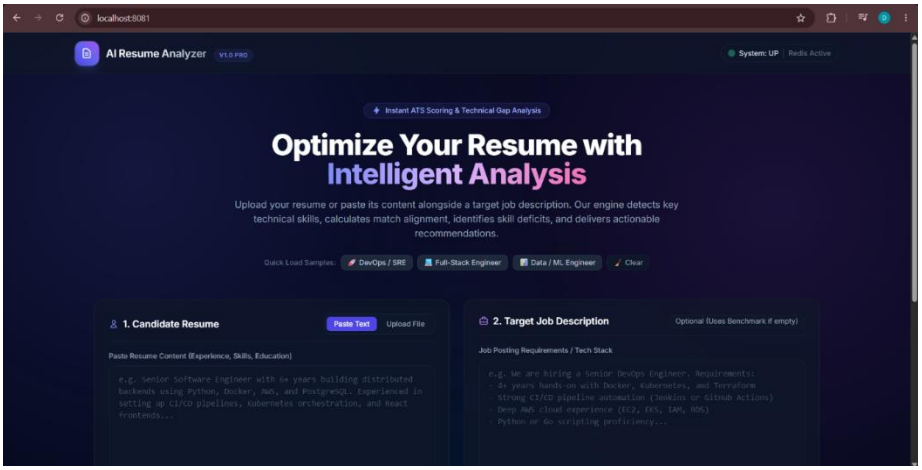


Figure 5.74: Application before the change: landing page heading at http://localhost:8081

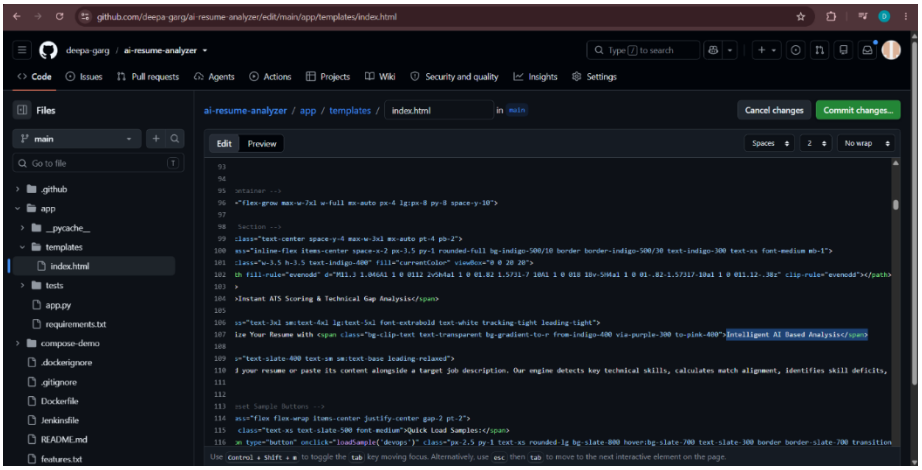


Figure 5.75: Editing templates/index.html in the GitHub repository

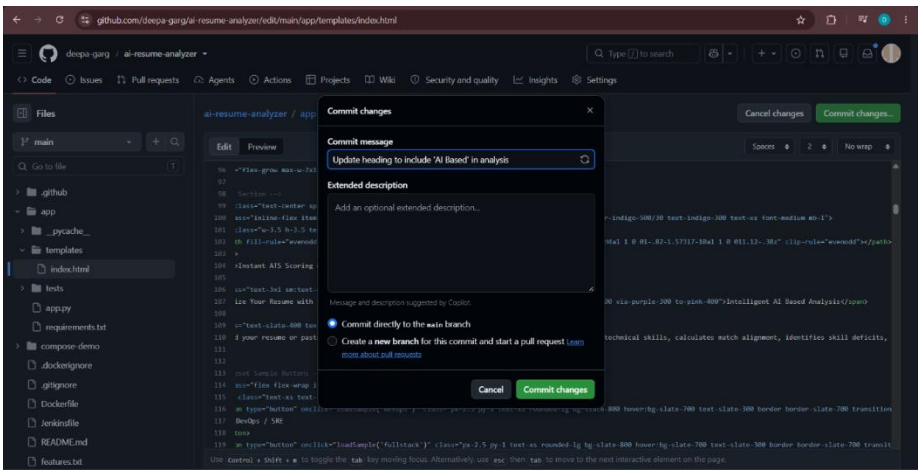


Figure 5.76: Committing the change directly to main with the message Update heading to include 'AI Based' in analysis

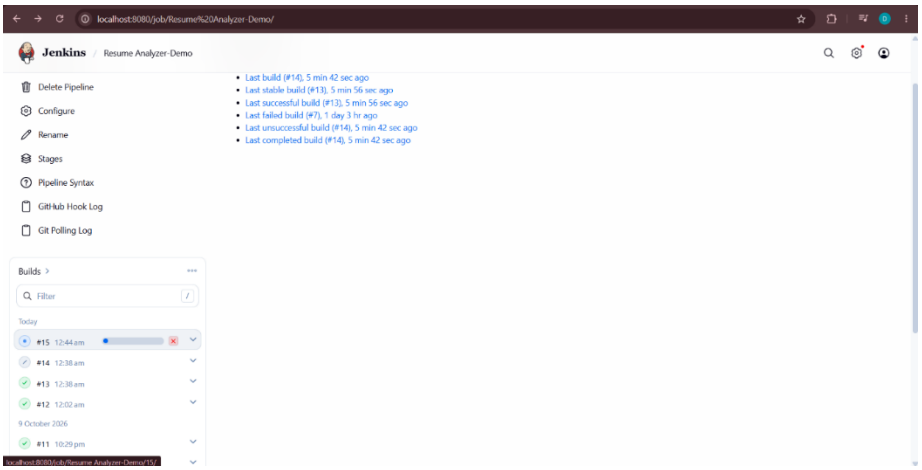


Figure 5.77: Jenkins job page: new builds appearing automatically in Build History after the push

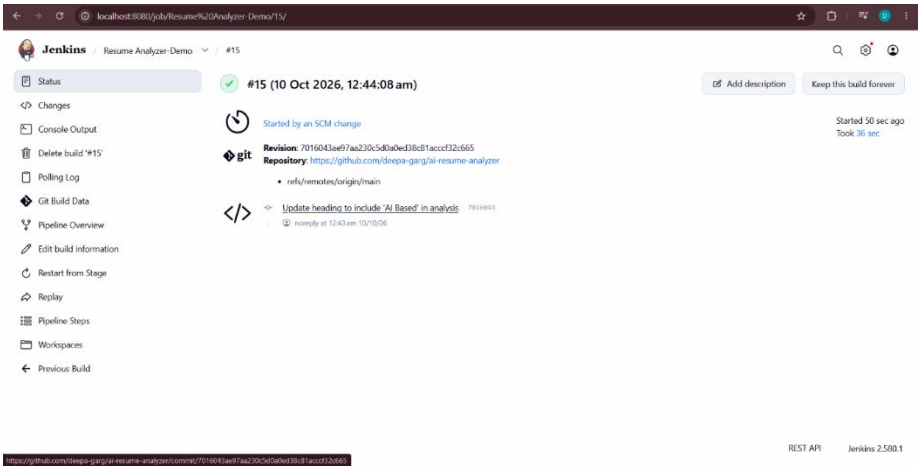


Figure 5.78: Build #15 started by an SCM trigger, showing the revision and the commit message

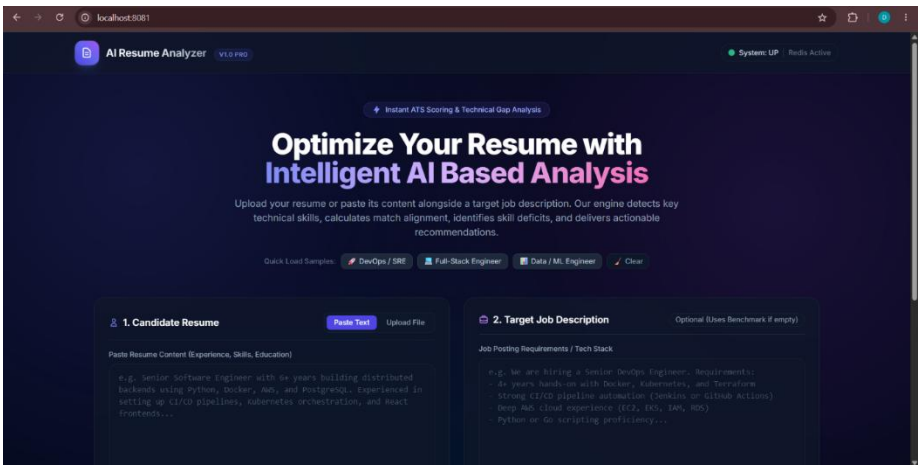


Figure 5.79: Application after the automatic build: updated heading Intelligent AI Based Analysis

5.5.4 Conclusion

Jenkins was integrated with GitHub by configuring Pipeline script from SCM and enabling the GitHub hook trigger. Jenkins fetched the Jenkinsfile, checked out the latest commit and ran the Build, Test and Deploy stages to SUCCESS.

5.6 Agile Project Lifecycle with Jira and DevOps Integration (Experiment 6)

5.6.1 Aim

To study and implement the Agile Software Project lifecycle using Jira with DevOps Pipeline integration (LO1: apply Agile methodologies and use Jira for sprint planning, task tracking and integrating with the DevOps workflow). A Jira Scrum project holds the work items of the AI Resume Analyzer, the GitHub repository is linked to Jira with GitHub for Jira, commits and pull requests carry the Jira issue key, Jira automation moves issues across the board, and GitHub Actions runs the tests.

5.6.2 Theory

Agile delivers working software in small increments, welcomes changing requirements and relies on frequent feedback. Scrum, the most widely used Agile framework, works in fixed-length sprints taken from an ordered Product Backlog; its roles are the Product Owner, the Scrum Master and the Developers, and its events are Sprint Planning, the Daily Scrum, the Sprint Review and the Sprint Retrospective. Requirements are written as user stories (As a <role>, I want <capability>, so that <benefit>) and estimated in story points; technical work is a task, a defect is a bug, and related items are grouped under an epic.

Jira is the Atlassian tool used to plan and track this work: a Scrum project provides a backlog, sprints and a board (To Do, In Progress, In Review, Done), and every work item has a key such as ARA-14. The GitHub for Jira app links a GitHub account to Jira, so that a branch name, commit message or pull request title containing an issue key appears in the Development panel of that issue (a smart commit). Jira Automation rules react to events such as Pull request created or Pull request merged by moving the issue to In Review or Done. GitHub Actions is the

continuous integration service of GitHub; a workflow in .github/workflows runs the tests on every push and pull request.

5.6.3 Implementation

The work items of the lab file (SCRUM-4, SCRUM-5 and SCRUM-6) are replaced here by the keys that Jira generated for this project, which start with ARA. The project backlog is shown in the table below. The echo commands that append to requirements.txt were run in Command Prompt or Git Bash (see Section 5.1.4).

Key	Summary
ARA-5	Build analyze endpoint
ARA-6	Build upload page
ARA-7	Write tests
ARA-8	Create Jenkins pipeline
ARA-9	Create GitHub Actions workflow
ARA-10	Dockerize app
ARA-11	Compose services
ARA-12	Provision EC2 with Terraform
ARA-13	Setup Nagios

Table 5.5: Product backlog of project ARA

Deepa Garg: Steps 1 to 5 (space, backlog, sprint, repository, integration)

Step 1: Jira space. A free Atlassian account was created and a Jira Cloud site was set up with the Software Development (Scrum) template. The team space was named AI Resume Analyzer, with the project key ARA. The board uses the standard To Do, In Progress, In Review and Done workflow, and Kunal Dodechani and Mohit Gabra were invited with the Member role.

Step 2: backlog and sprint. Work items were created in the backlog and grouped under an epic, each was assigned to a member, and a sprint was created, filled with items and started. The items then appeared on the Scrum board under To Do. The board is used throughout the experiment: an item moves to In Review when a pull request is opened and to Done when it is merged, and the sprint is completed at the end with the Complete sprint button.

Step 3: local project. The Python application was scaffolded and run locally to confirm it works, and a .gitignore file (.venv/, __pycache__/, *.pyc) was committed.

```
cd ai-resume-analyzer
python -m venv .venv
.venv\Scripts\activate
pip install -r app\requirements.txt
python app\app.py
# open http://localhost:5000/health, then press Ctrl+C
```

```
git checkout main
git add .gitignore
git commit -m "Add .gitignore"
git push origin main
```

Step 4: repository and collaborators. The public repository ai-resume-analyzer from Experiment 1 was confirmed, and Kunal Dodechani and Mohit Gabra were confirmed under Settings, Collaborators.

Step 5: GitHub–Jira integration. The GitHub for Jira app was installed from Apps in Jira, the GitHub account was connected, and the app was given access to the repository ai-resume-analyzer. Jira confirmed that the account was connected, so that development activity is reflected on Jira issues.

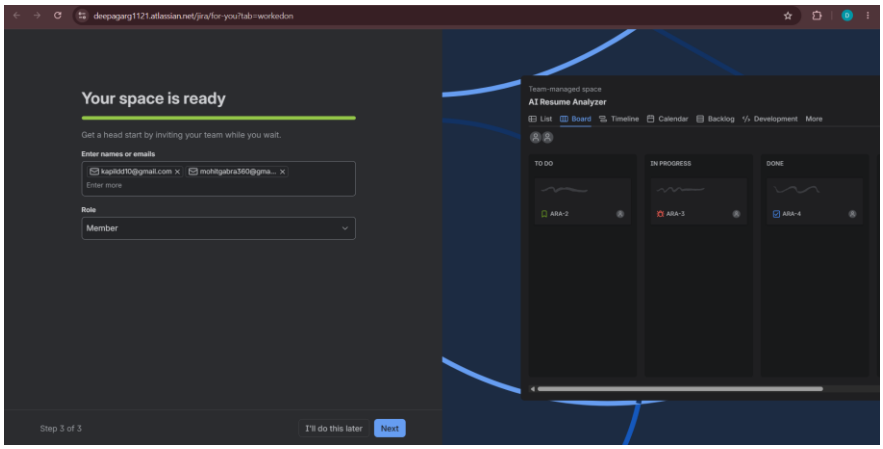


Figure 5.80: Jira space ready, with the board and the invitation of team members

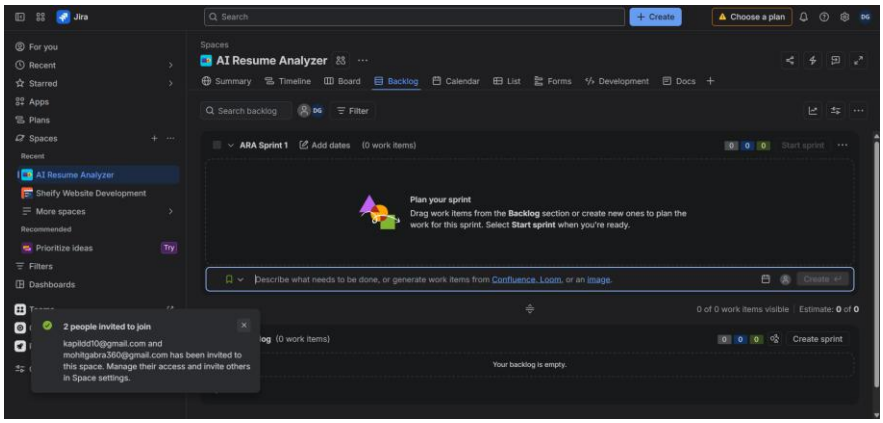


Figure 5.81: Jira backlog with Sprint 1 created and two people invited to the space

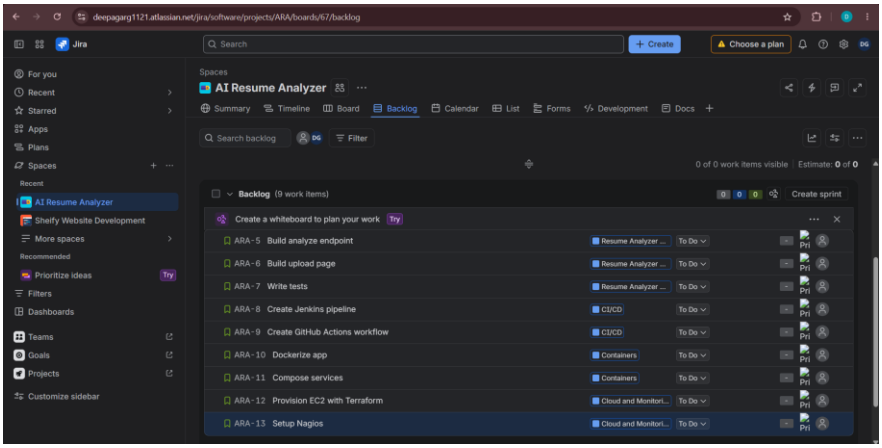


Figure 5.82: Product backlog with the work items of the project

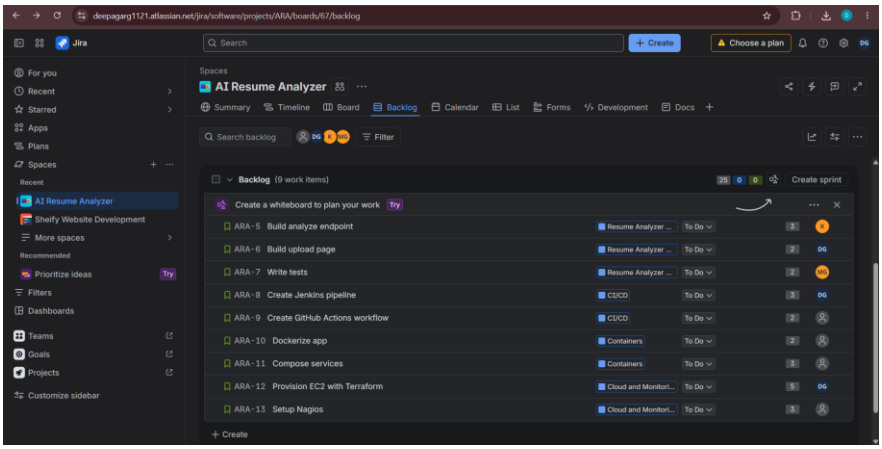


Figure 5.83: Backlog items assigned to team members

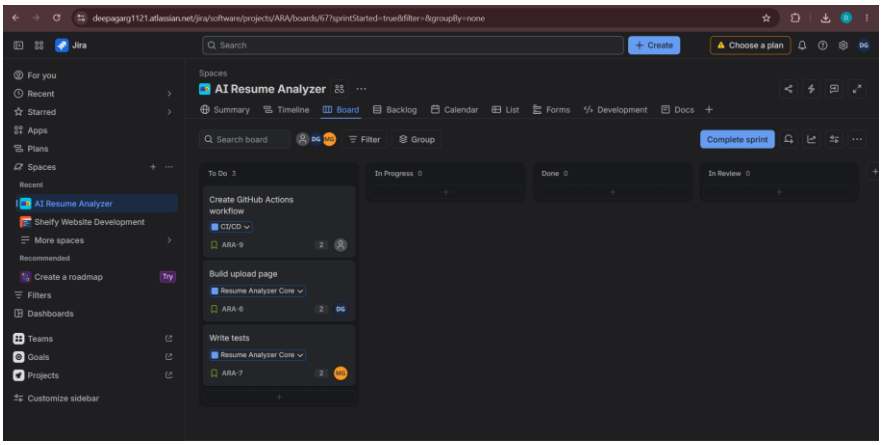


Figure 5.84: Scrum board after starting the sprint, with the Complete sprint button

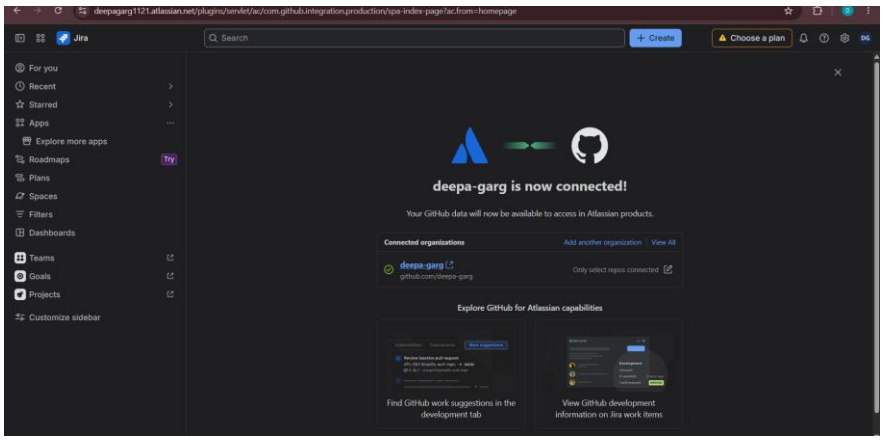


Figure 5.85: GitHub for Jira connected to the GitHub account

Kunal Dodechani: Steps 6 to 8 (smart commit, rule 1, pull request)

Step 6: smart commit. A branch whose name contains the issue key was created from main, a change was committed with a message that begins with the issue key, and the branch was pushed. The Development panel of the issue then showed the linked commit.

```
git checkout main
git pull origin main
git checkout -b SCRUM-5-resume-upload
echo "Resume upload page: users upload a resume for analysis" >> README.md
git add .
git commit -m "SCRUM-5 Add resume upload feature description"
git push origin SCRUM-5-resume-upload
```

Step 7: automation rule 1. In Project settings, Automation, a rule was created with the trigger Pull request created and the action Transition work item to In Review, named PR created - In Review and turned on. **Step 8: pull request.** On GitHub the pull request was opened with the title SCRUM-5 Resume upload feature, and the issue moved to In Review by itself.

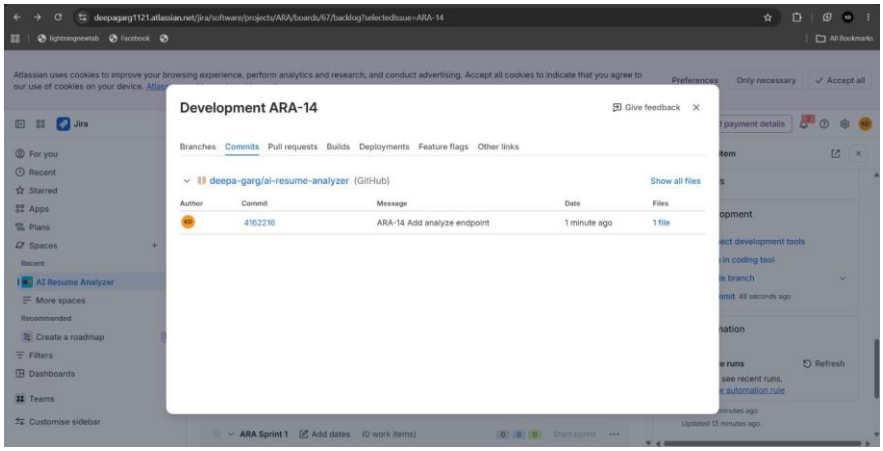


Figure 5.86: Development panel of issue ARA-14 showing the linked commit

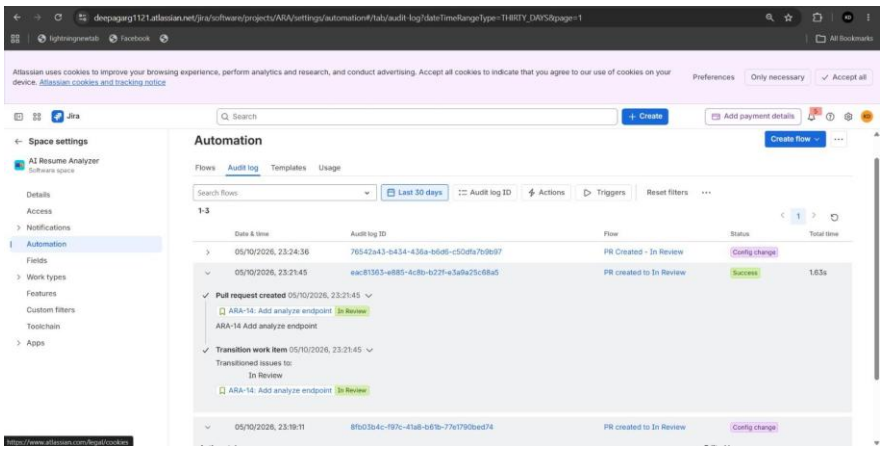


Figure 5.87: Jira automation audit log: pull request created moves the issue to In Review

Mohit Gabra: Steps 7, 8 and 11 (rule 2, review and merge, second branch)

Step 7: automation rule 2. A second rule was created with the trigger Pull request merged and the action Transition work item to Done, named PR merged - Done. **Step 8: review and merge.** The pull request was opened, Files changed was reviewed, Approve was chosen under Review changes, and the pull request was merged; the issue moved to Done automatically.

Step 11, part 1: second branch. To create a realistic merge conflict, a second branch was created from main before the CI branch was merged, and a different line was appended to the last line of app/requirements.txt. **Part 2:** after the CI branch was merged into main, main was merged into the second branch. Both branches had added a last line to the same file, so Git reported a merge conflict. The three marker lines were removed, both lines (pytest and pytest-cov) were kept, and the result was committed and pushed. The conflict message then disappeared from the pull request.

```
git checkout main
git pull origin main
git checkout -b SCRUM-6-unit-tests
echo pytest-cov >> app/requirements.txt
git add .
git commit -m "SCRUM-6 Add unit tests for resume analyzer"
git push origin SCRUM-6-unit-tests

git fetch origin
git merge origin/main
# edit app/requirements.txt: remove <<<<<<<, ======= and >>>>>>>, keep both lines
git add app/requirements.txt
git commit -m "SCRUM-6 Resolve merge conflict in requirements.txt"
git push origin SCRUM-6-unit-tests
```

Deepa Garg: Steps 9, 10 and 12 (GitHub Actions and completion)

Step 9: GitHub Actions. A branch was created for the CI task, pytest was added to app/requirements.txt, unit tests were written in app/tests, and a workflow file .github/workflows/ci.yml was added. The workflow runs on a push or pull request to main, sets up Python 3.11, installs the requirements and runs pytest.

```
git checkout main
git pull origin main
git checkout -b SCRUM-4-github-actions-ci
echo pytest >> app/requirements.txt
pip install pytest
python -m pytest app/tests
```

```
name: CI
on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - run: pip install -r app/requirements.txt
      - run: python -m pytest app/tests
```

```
git add .
git commit -m "SCRUM-4 Add GitHub Actions CI workflow and unit tests"
git push origin SCRUM-4-github-actions-ci
```

Step 10: verification. A pull request was opened for the CI branch, the checks at the bottom of the pull request showed that all checks had passed, and the pull request was merged into main. **Step 12: final state.** Main was pulled and the history was shown with git log --oneline --graph --all, GitHub Insights, Contributors was opened to show each member's contributions, all sprint items were in Done on the board, and Complete sprint was clicked to finish the sprint.

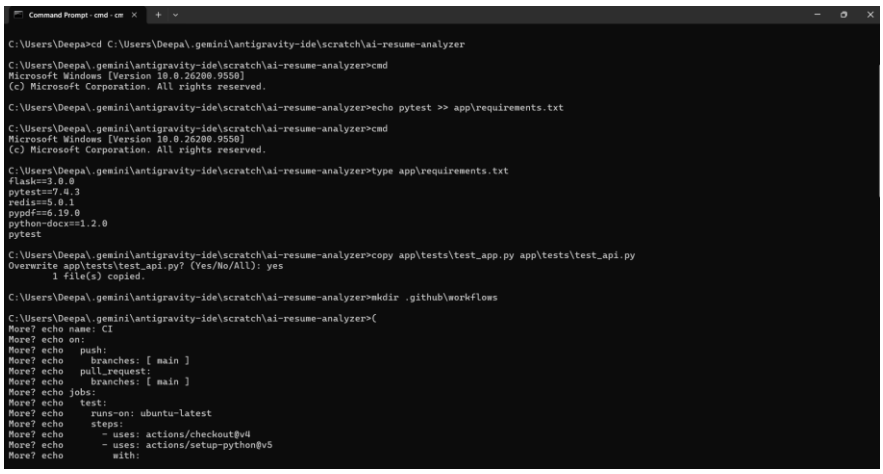


Figure 5.88: Adding pytest to requirements.txt, copying the tests and creating the workflow file

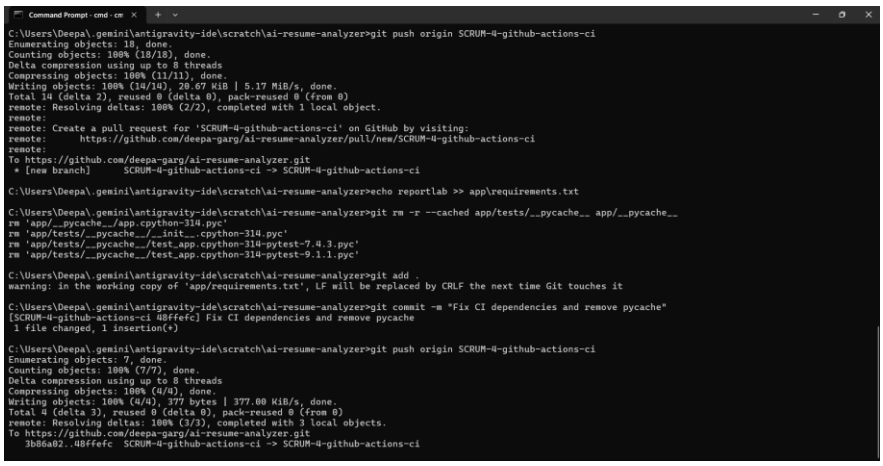


Figure 5.89: Committing the fix and pushing the CI branch to GitHub

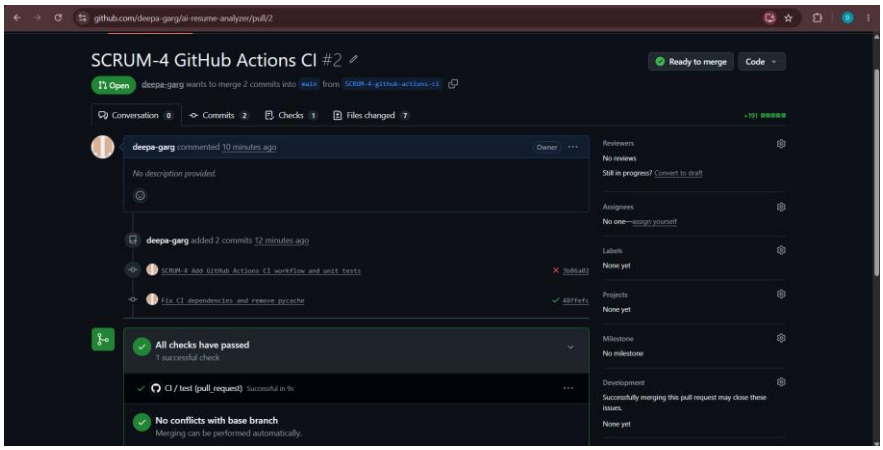


Figure 5.90: Pull request with the GitHub Actions check passed

Definition of Done

A work item is moved to Done only when all of the following hold:

1. The branch name or the commit message contains the Jira issue key, so the work is linked to the ticket.
2. The change reached main through a pull request, and the GitHub Actions check on that pull request is green.
3. The pull request was reviewed and approved by a member other than its author.
4. The merge of the pull request moved the issue to Done through the Jira automation rule.

5.6.4 Conclusion

Integrating Jira with GitHub established an automated Agile development lifecycle for the AI Resume Analyzer. The backlog was organised into a sprint, issue keys linked code to tickets, and GitHub Actions and Jira Automation removed manual effort by testing code and moving tickets on pull requests and merges.

5.7 Monitoring with Nagios (Experiment 7)

5.7.1 Aim

To implement monitoring using Prometheus / Grafana / Nagios; Nagios is used **(LO5).** The Nagios Core server runs in a Docker container on Deepa Garg's PC and monitors a Windows host (Kunal Dodechani's PC) through NSClient++.

5.7.2 Theory

Monitoring continuously checks that systems and services are healthy, so that failures are noticed before users complain, and closes the DevOps feedback loop. Nagios Core is configured with objects: a host is a machine to monitor, a service is a check on a host (PING, HTTP, disk space), and every check returns OK, WARNING, CRITICAL or UNKNOWN (UP or DOWN for hosts). Remote hosts are checked by network tests or through an agent: NRPE lets the server ask an agent to run a check, and NSClient++ is the Windows agent with an NRPE server that accepts requests only from its allowed hosts and needs port 5666 open in the firewall.

Nagios Core does not install natively on Windows, so it is run in a container with Docker Desktop (WSL 2 backend), and Windows hosts are monitored with NSClient++. Port 8090 is used for the Nagios web interface because Jenkins already uses 8080.

5.7.3 Implementation

Deepa Garg: Steps 1 to 4 and 7

```
docker --version
wsl --install
wsl --set-default-version 2

docker pull jasonrivers/nagios:latest
docker run --name nagios4 -d -p 8090:80 jasonrivers/nagios:latest
docker ps

docker exec -it nagios4 htpasswd -c /opt/nagios/etc/htpasswd.users nagiosadmin

# Web interface: http://localhost:8090/nagios/  (user nagiosadmin)

docker exec -it nagios4 /opt/nagios/libexec/check_nrpe -H <Kunal-IP>        # Step 7
```

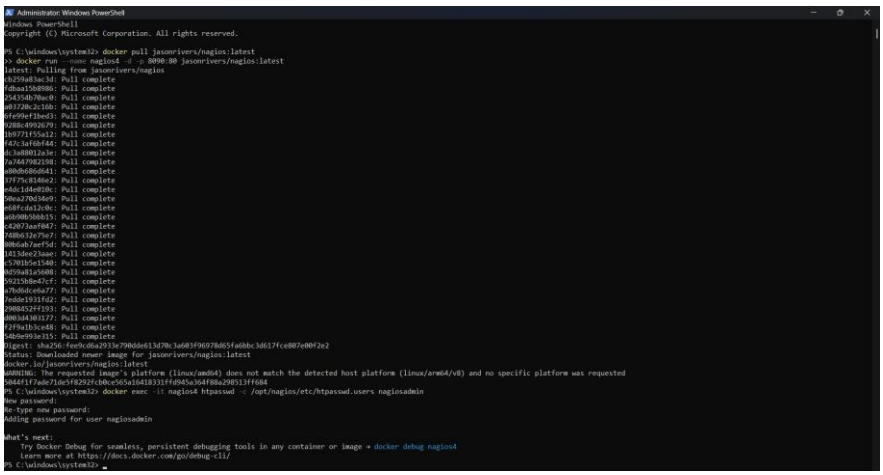


Figure 5.91: Pulling and running the nagios4 container and setting the nagiosadmin password

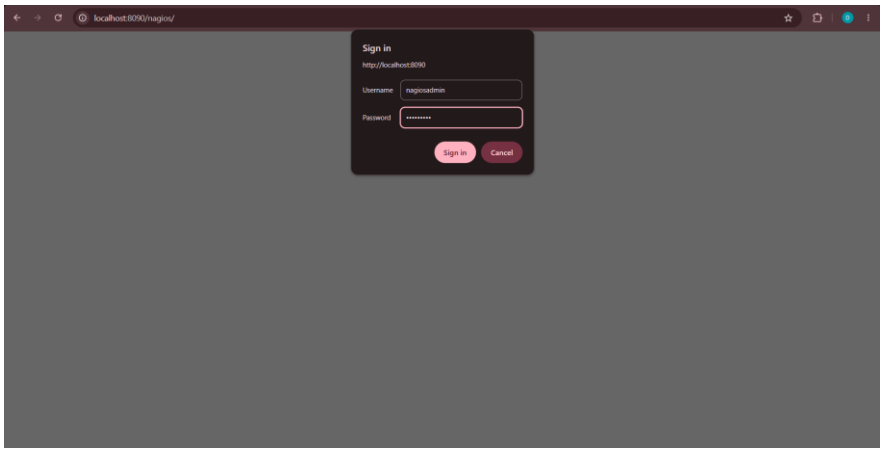


Figure 5.92: Nagios sign-in screen

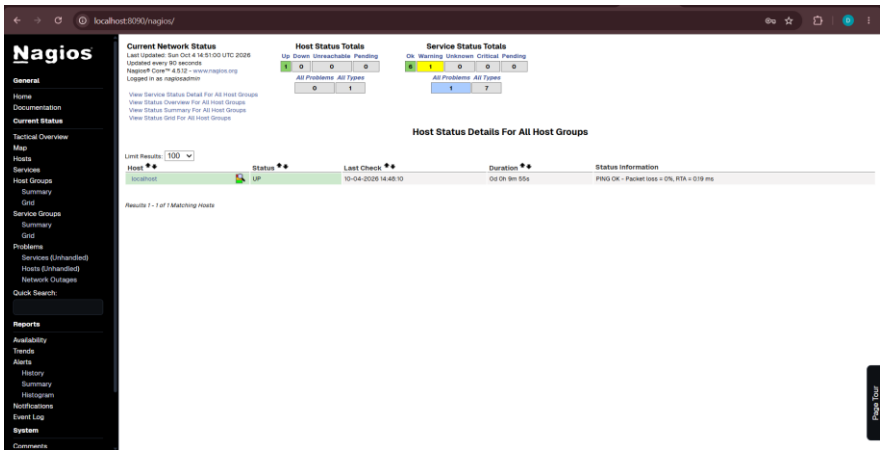


Figure 5.93: Nagios host status page after login

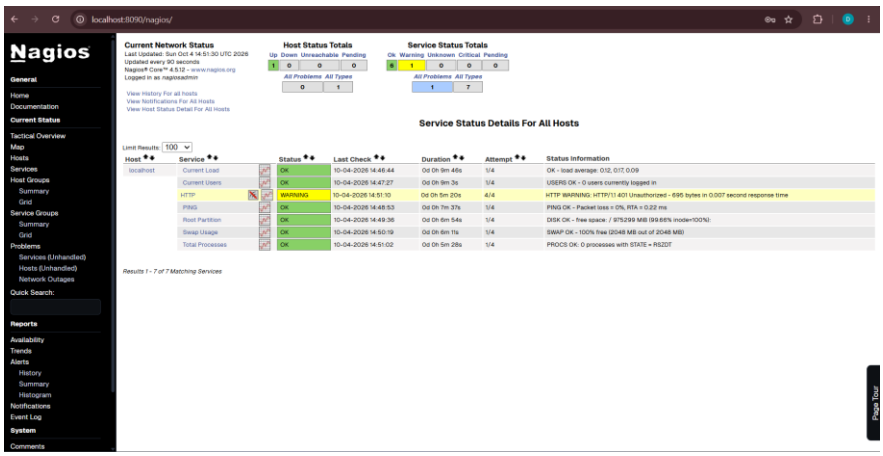


Figure 5.94: Nagios service status details

Kunal Dodechani: Steps 5 and 6

```
Invoke-WebRequest -Uri
"https://github.com/mickem/nscp/releases/latest/download/NSCP-0.5.2-x64.msi" -
OutFile "NSCP.msi"
Start-Process msiexec.exe -ArgumentList '/i NSCP.msi ADDLOCAL=ALL
MODULES=CheckExternalScripts,NRPEServer /qn' -Wait

notepad "C:\Program Files\NSClient++\nsclient.ini"
# [/settings/default]
# allowed hosts=127.0.0.1,<Deepa-IP>
Restart-Service nscp

New-NetFirewallRule -DisplayName "NRPE 5666" -Direction Inbound -LocalPort 5666
-Protocol TCP -Action Allow
```

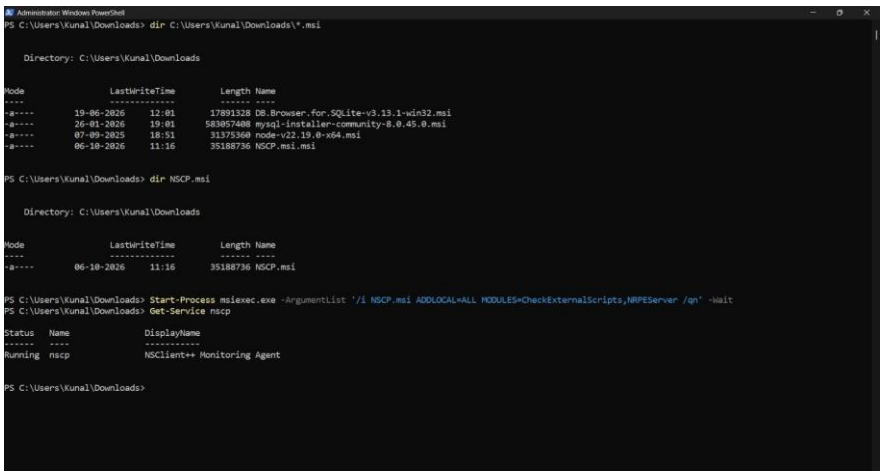


Figure 5.95: Downloading and installing NSClient++ and checking that the nscp service is running

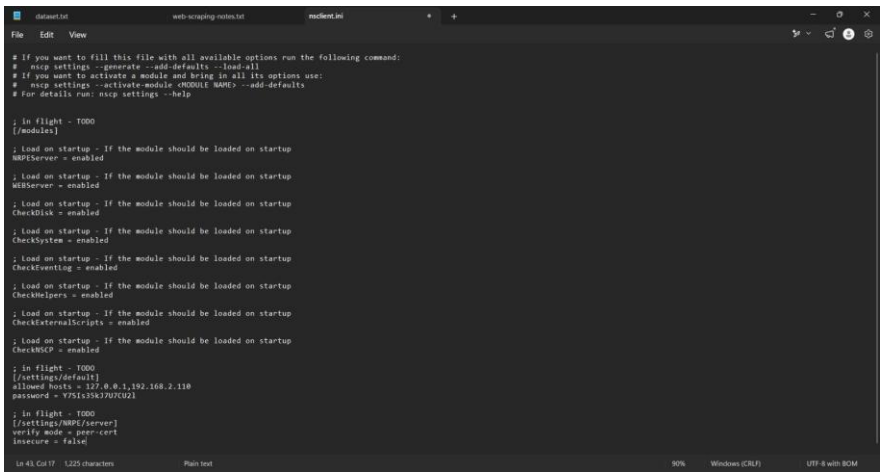


Figure 5.96: nsclient.ini with the NRPE server enabled and the allowed hosts line

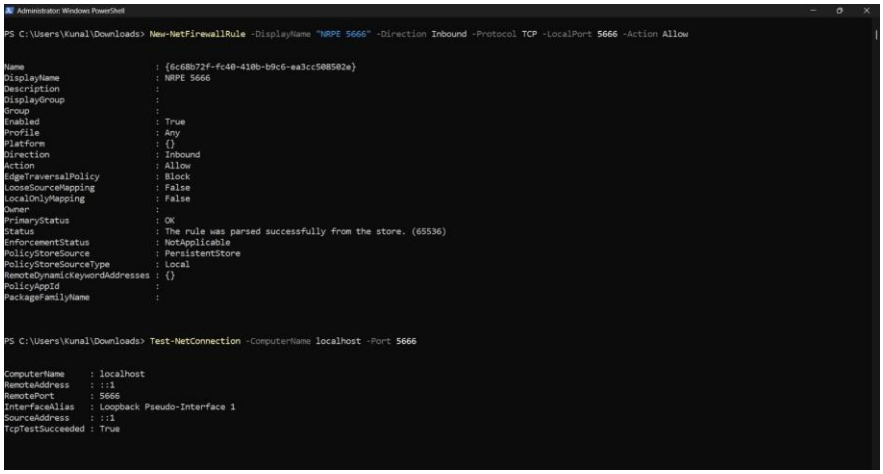


Figure 5.97: Firewall rule for port 5666 and connection test

Mohit Gabra: Steps 8 and 9

```
docker exec -it nagios4 sh -c "vi /opt/nagios/etc/servers/ResumeAnalyzerHost001.cfg"

define host {
   use         windows-server
   host_name   ResumeAnalyzer_Host_001
   alias       Resume Analyzer Host 001
   address     <Kunal-IP>
   register    1
}

define service {
   host_name           ResumeAnalyzer_Host_001
   service_description PING
   check_command       check_ping!100.0,20%!500.0,60%
   max_check_attempts  2
   check_interval      2
   retry_interval      2
   check_period        24x7
   contact_groups      admins
```

```
    notification_period 24x7
    register            1
}

docker exec -it nagios4 /opt/nagios/bin/nagios -v /opt/nagios/etc/nagios.cfg
docker restart nagios4
```

The configuration check should end with Total Errors: 0. After the restart, the host appears under Current Status, Hosts and the PING service under Services within about two minutes.

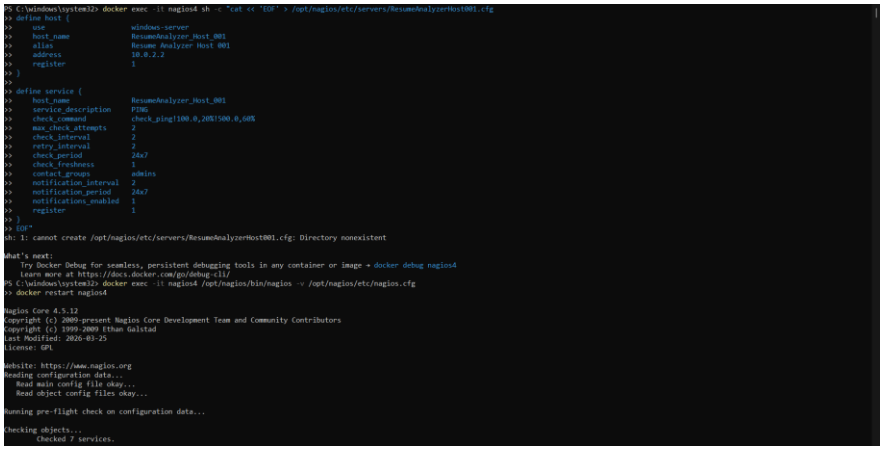


Figure 5.98: Creating the host and PING service definition file inside the Nagios container

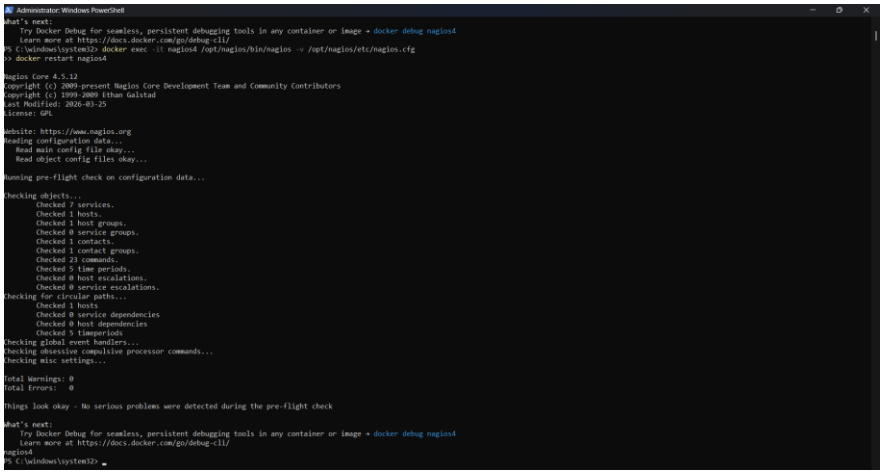


Figure 5.99: Nagios configuration check (Total Errors: 0) and restart of the container

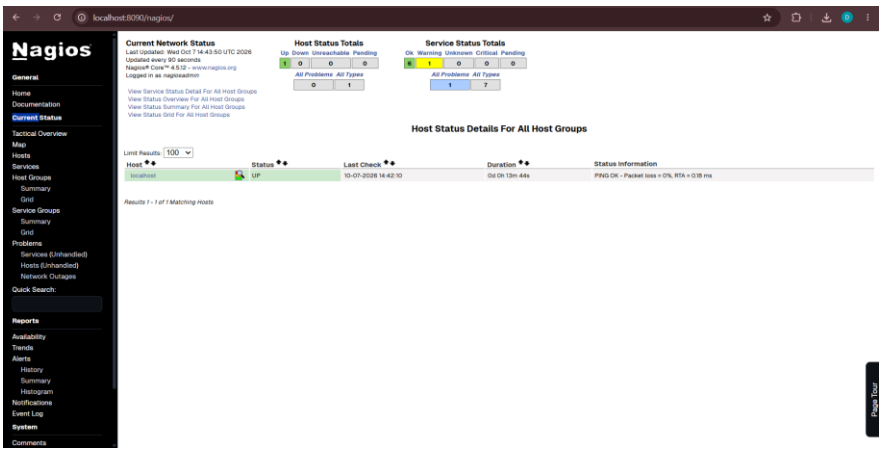


Figure 5.100: Current Status, Hosts: the monitored host is UP

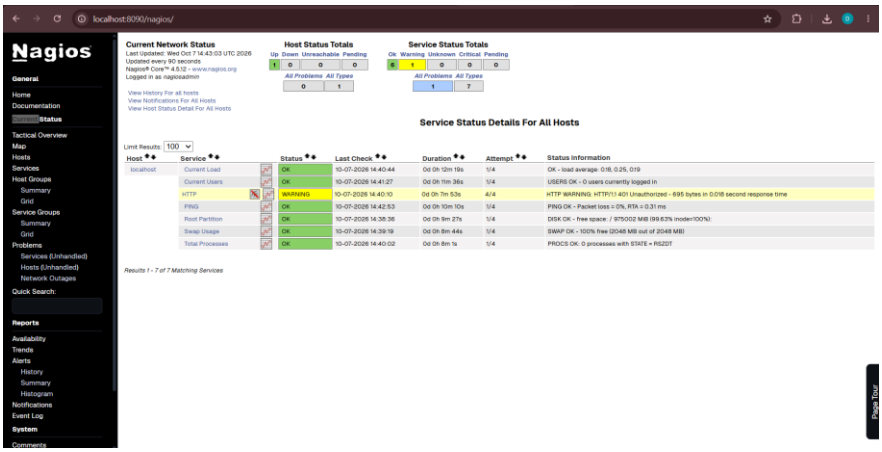


Figure 5.101: Service status details for the monitored host

5.7.4 Conclusion

Nagios was implemented as a monitoring tool for hosts and services. The system performed continuous checks and displayed the status of the monitored resources, helping identify infrastructure and service issues.

6. Testing

6.1 Automated Unit Tests

The tests use pytest and the Flask test client, so they run without starting a server. They are kept in app/tests. The first checks that GET /health returns HTTP 200, and the second checks that the home page loads.

```
import os
import sys
sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
from app import app

def test_health():
    client = app.test_client()
    response = client.get("/health")
    assert response.status_code == 200
```

The tests are run locally with python -m pytest app/tests, and by GitHub Actions on every push and pull request to main (Section 5.6).

6.2 Container and Service Checks

Check	How it was verified	Expected result
Containerised application	docker run, then http://localhost:5000/health	HTTP 200 with status UP
Compose stack	docker compose ps; http://localhost:8081 and http://localhost:8082	All services running; application through nginx; phpMyAdmin login page
Reverse proxy	Request to nginx on port 8081	Response from the Flask application
Jenkins pipelines	Build Now, Console Output	Finished: SUCCESS
Monitoring	Nagios Current Status, Hosts and Services	Host UP and PING service OK

Table 6.1: Checks performed on the deployed system

6.3 Pipeline Verification

Pipeline verification is done at three levels: the pull request check from GitHub Actions (green run), the Jenkins build triggered by the push (Build, Test and Deploy stages completed), and the board state in Jira (items moved to In Review and Done by the automation rules).

7. Results and Discussion

7.1 Outcomes against Objectives

Objective	Outcome
Match analysis of a resume against a job description	Achieved: score, matched and missing skills, skill matrix and recommendations (Chapter 4)
Jira Scrum planning linked to code	Achieved: backlog and sprint in project ARA; GitHub for Jira connected; automation rules (Section 5.6)
Git and GitHub workflow	Achieved: branches, merges, conflict resolution, reset, revert, restore (Section 5.1)
Docker image and container	Achieved: resume-analyzer:v1 running on port 5000 (Section 5.2)
Docker Compose stack	Achieved: nginx, app, MySQL, Redis and phpMyAdmin from one file (Section 5.3)
Jenkins pipelines	Achieved: Declarative and Scripted pipelines, Jenkinsfile from GitHub (Sections 5.4, 5.5)

Objective	Outcome
Monitoring	Achieved: Nagios host and services visible; NSClient++ configured (Section 5.7)

Table 7.1: Objectives and their outcomes

7.2 Observations

- Putting the issue key in the branch name and commit message made the Jira Development panel and the automation rules work without any manual update of the board.
- Running Nagios in a container made the monitoring server easy to start and discard, and the service status page showed an HTTP warning with a 401 response when the web interface was checked without credentials, which illustrates the difference between a service that is up and one that answers as expected.
- The cache health information (the mode and whether Redis is connected) in /health makes the application easy to check from Docker, Jenkins and Nagios.

7.4 Limitations

- The analysis is keyword-based, so a resume that expresses a skill in unusual wording may not be matched.
- The Deploy stage of the Jenkins pipeline uses echo steps, as in the lab experiments, and does not yet start the containers.
- The monitored host is checked with PING; application-level checks such as /health are an optional extension.

8. Conclusion and Future Scope

8.1 Conclusion

The AI Resume Analyzer was delivered through a complete Agile and DevOps flow. The work was planned in Jira, versioned in Git and GitHub, packaged with Docker and Docker Compose, built and deployed through Jenkins, tested by GitHub Actions and monitored by Nagios. Each of the three members performed part of every experiment and produced evidence of it. The project shows that these practices are most valuable together: each tool takes its input from the previous one, and the issue key, the commit, the pull request, the build and the monitoring check all refer to the same unit of work.

8.2 Future Scope

- Replace the echo steps of the Jenkins Deploy stage with real steps that build the image and run docker compose up -d, with a smoke test and automatic rollback.
- Add application-level monitoring in Nagios with check_http against /health, and add Prometheus and Grafana for metrics.
- Replace the keyword matcher with semantic matching using language models, and extend the skill taxonomy beyond technical roles.

- Store analysis history in the MySQL service and add user accounts.
- Provision the infrastructure as code (for example with Terraform), as planned in the backlog.

References

1. S. Chacon and B. Straub, Pro Git, 2nd ed. Apress, 2014. https://git-scm.com/book
2. Docker Inc., Docker Documentation. https://docs.docker.com
3. Docker Inc., Docker Compose Documentation. https://docs.docker.com/compose
4. Jenkins Project, Jenkins User Documentation and Pipeline Syntax. https://www.jenkins.io/doc
5. GitHub, GitHub Docs: Actions, Webhooks and Pull Requests. https://docs.github.com
6. Atlassian, Jira Software Documentation and Agile Coach. https://www.atlassian.com/agile
7. K. Schwaber and J. Sutherland, The Scrum Guide. https://scrumguides.org
8. Nagios Enterprises, Nagios Core Documentation. https://www.nagios.org/documentation
9. NSClient++ Documentation. https://nsclient.org
10. Pallets, Flask Documentation. https://flask.palletsprojects.com
11. pytest Documentation. https://docs.pytest.org

Appendix A: Running the Project

A.1 Run locally

```
cd ai-resume-analyzer
python -m venv .venv
.venv\Scripts\activate
pip install -r app\requirements.txt
python app\app.py
# open http://localhost:5000/health
```

A.2 Run as a container

```
docker build -t resume-analyzer:v1 .
docker run -d --name resume-app -p 5000:5000 resume-analyzer:v1
```

A.3 Run the full stack with Docker Compose

```
cd compose-demo
docker compose up -d
# application:  http://localhost:8081
# phpMyAdmin:   http://localhost:8082  (server database, user root)
docker compose down
```

A.4 Run the tests

```
pip install pytest
python -m pytest app/tests
```

A.5 Start Jenkins and Nagios

```
cd C:\jenkins
java -jar jenkins.war                # http://localhost:8080

docker run --name nagios4 -d -p 8090:80 jasonrivers/nagios:latest   #
http://localhost:8090/nagios/
```

Lab Outcomes

A. LO1 – Apply Agile methodologies and use Jira for sprint planning, task tracking and integrating with the DevOps workflow (Experiment 6).

B. LO2 – Perform version control operations using Git and apply collaboration workflows (Experiment 1).

C. LO3 – Build CI/CD pipelines using Jenkins with Declarative and Scripted syntax, and automate application deployment using GitHub and Jenkins (Experiments 4 and 5).

D. LO4 – Containerize, orchestrate and deploy applications using Docker, Docker Compose, Kubernetes and cloud infrastructure tools (Experiments 2 and 3).

E. LO5 – Implement monitoring of hosts and services using Nagios (Experiment 7).